

CSCE 5300: Introduction to Big Data and Data Science

Chapter 5 (5.2): Data Learning Methods



Outline



- **5.2.1. K-Nearest Neighbors**
- **5.2.2. Decision Tree**
- **5.2.3. Linear Regression**
- **5.2.4. Logistic Regression**
- **5.2.5. Support Vector Machines**
- **5.2.6. Time Series**
- **5.2.7. Random Forest Algorithm**

5.2.1. K- Nearest Neighbors



- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on **supervised Learning technique**.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

5.2.1. K- Nearest Neighbors

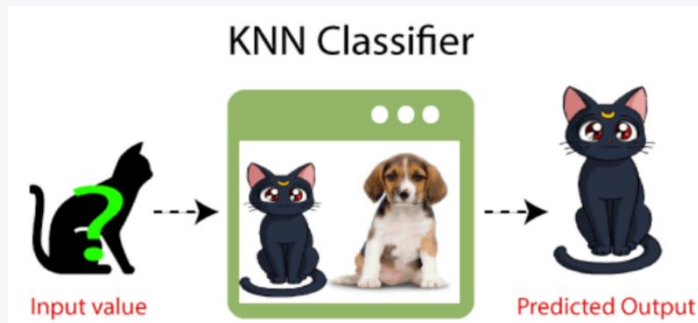


- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

5.2.1. K- Nearest Neighbors

- **Example:** Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog.
- So, for this identification, we can use the KNN algorithm, as it works on a similarity measure.
- Our KNN model will find the similar features of the new data set to the cat and dog images and based on the most similar features it will put it in either cat or dog category (Fig. 5.2.1).

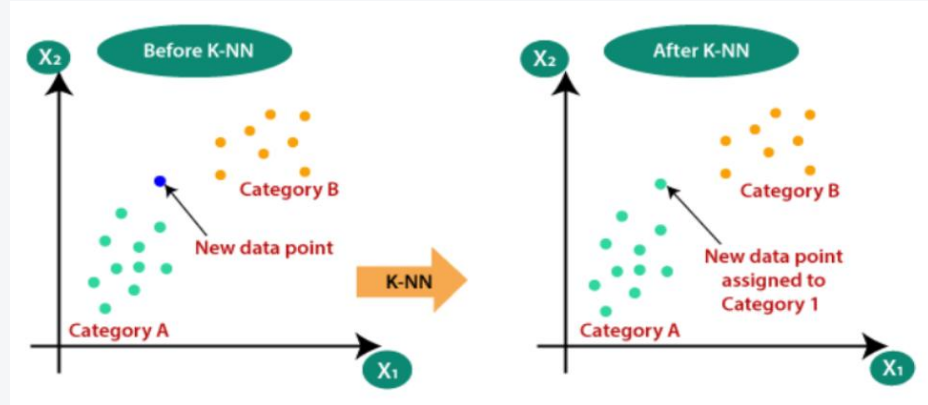
Fig. 5.2.1.



5.2.1. K- Nearest Neighbors

- Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x_1 , so this data point will lie in which of these categories.
- To solve this type of problem, we need a K-NN algorithm.
- With the help of K-NN, we can easily identify the category or class of a particular dataset.
- Consider the below diagram (Fig. 5.2.2):

Fig. 5.2.2.



5.2.1. K- Nearest Neighbors



- **Algorithm of K-Nearest Neighbors**
- **Step 1** – For implementing any algorithm, we need dataset. So, during the first step of KNN, we must load the training as well as test data.
- **Step 2** – Next, we need to choose the value of K i.e. the nearest data points. K can be any integer.

5.2.1. K- Nearest Neighbors

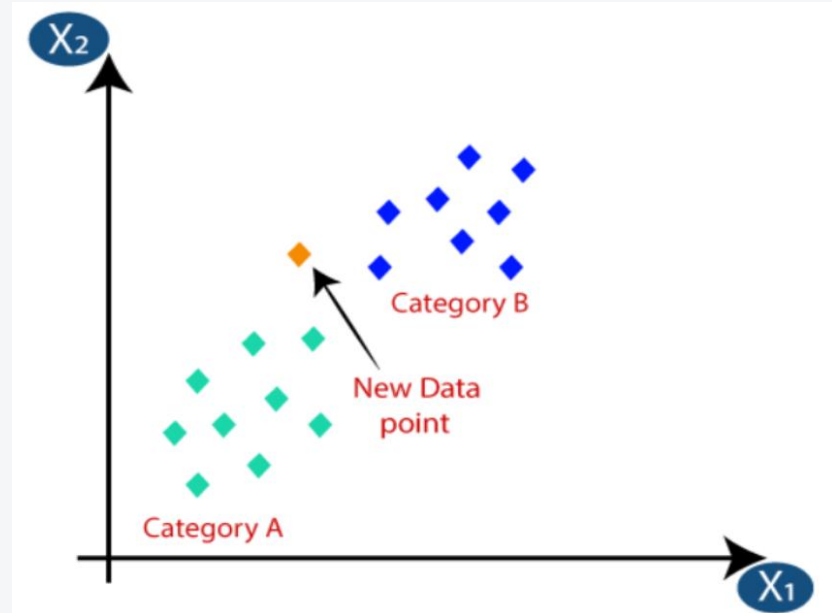


- **Step 3** – For each point in the test data do the following –
 - **3.1** – Calculate the distance between test data and each row of training data with the help of any of the method namely: Euclidean, Manhattan or Hamming distance. The most commonly used method to calculate distance is Euclidean.
 - **3.2** – Now, based on the distance value, sort them in ascending order.
 - **3.3** – Next, it will choose the top K rows from the sorted array.
 - **3.4** – Now, it will assign a class to the test point based on most frequent class of these rows.
- **Step 4** – End

5.2.1. K- Nearest Neighbors

- Suppose we have a new data point, and we need to put it in the required category.
- Consider the below image (Fig. 5.2.3):

Fig. 5.2.3.

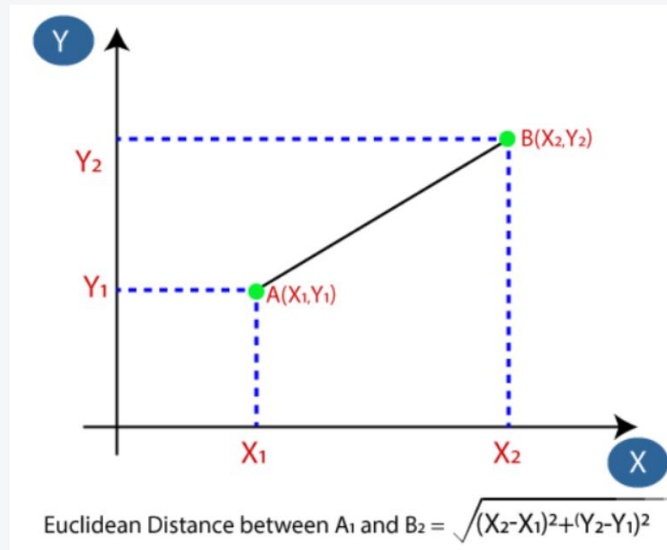


5.2.1. K- Nearest Neighbors



- Firstly, we will choose the number of neighbors, so we will choose the $K=5$.
- Next, we will calculate the **Euclidean distance** between the data points.
- The Euclidean distance is the distance between two points, which we have already studied in geometry.
- It can be calculated as (Fig. 5.2.4):

Fig. 5.2.4.



5.2.1. K- Nearest Neighbors



- By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B.
- Consider the below image (Fig. 5.2.5):

Fig. 5.2.5.



5.2.1. K- Nearest Neighbors



- As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.
- How to select the value of K in the K-NN Algorithm?
- Below are some points to remember while selecting the value of K in the K-NN algorithm:
- There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them.
- A very low value for K such as $K=1$ or $K=2$, can be noisy and lead to the effects of outliers (in statistics, an outlier is a data point that differs significantly from other observations).
- Large values for K are good, but it may find some difficulties.

5.2.1. K- Nearest Neighbors



- For 1NN we assign each data point to the class of its closest neighbor.
-
- For KNN we assign each data point to the majority class of its K closest neighbors where K is a parameter.
- The rationale of KNN classification is that, based on the contiguity hypothesis, we expect a test data point d to have the same label as the training data point located in the local region surrounding d .

5.2.1. K- Nearest Neighbors



- Decision boundaries in 1NN are concatenated segments of the *Voronoi tessellation*.
- The Voronoi tessellation of a set of objects decomposes space into Voronoi cells, where each object's cell consists of all points that are closer to the object than to other objects.
- In our case, the objects are data points.
- The Voronoi tessellation partitions the plane into $|D|$ convex polygons, each containing its corresponding data point, where a convex polygon is a convex region in 2-dimensional space bounded by lines.

5.2.1. K- Nearest Neighbors



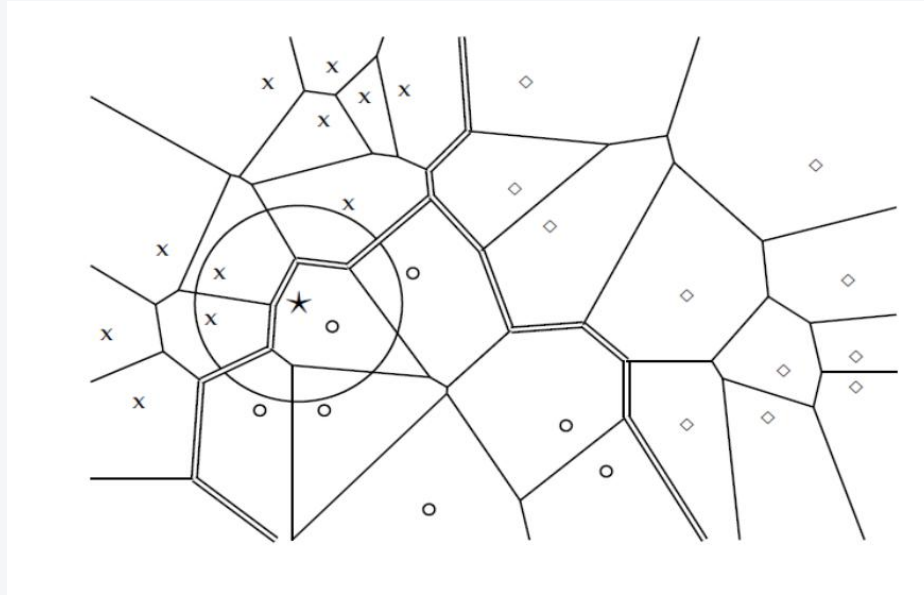
- For general $k \in \mathbf{N}$ in kNN, consider the region in the space for which the set of k nearest neighbors is the same.
- This again is a convex polygon, and the space is partitioned into convex polygons, within each of which the set of k nearest neighbors is invariant.
- 1NN is not very robust.
- The classification decision of each test document relies on the class of a single training data point, which may be incorrectly labeled or atypical.
- kNN for $k > 1$ is more robust.
- It assigns data points to the majority class of their k closest neighbors, with ties broken randomly

5.2.1. K- Nearest Neighbors



- There is a probabilistic version of this kNN classification algorithm.
- We can estimate the probability of membership in class c as the proportion of the k nearest neighbors in c .
- Fig. 5.2.6 gives an example for $k = 3$.
- Probability estimates for class membership of the star are $\hat{P}(\text{circle class} | \text{star}) = 1/3$, $\hat{P}(\text{X class} | \text{star}) = 2/3$, and $\hat{P}(\text{diamond class} | \text{star}) = 0$.
- The 3NN estimate ($\hat{P}_1(\text{circle class} | \text{star}) = 1/3$) and the 1NN estimate ($\hat{P}_1(\text{circle class} | \text{star}) = 1$) differ with 3nn preferring the X class and 1nn preferring the circle class .

5.2.1. K- Nearest Neighbors



- Fig. 5.2.6. Voronoi tessellation and decision boundaries (double lines) in 1NN classification.
- The three classes are: X, circle and diamond.

5.2.1. K- Nearest Neighbors



- The parameter k in kNN is often chosen based on experience or knowledge about the classification problem at hand.
- It is desirable for k to be odd to make ties less likely. $k = 3$ and $k = 5$ are common choices, but much larger values between 50 and 100 are also used.
- An alternative way of setting the parameter is to select the k that gives best results on a held-out portion of the training set.
- We can also weight the “votes” of the k nearest neighbors by their cosine similarity. In this scheme, a class’s score is computed as (in data analysis, cosine similarity is a measure of similarity between two non-zero vectors defined in an inner product space):

5.2.1. K- Nearest Neighbors



Given two n -dimensional **vectors** of attributes, **A** and **B**, the cosine similarity, $\cos(\theta)$, is represented using a **dot product** and **magnitude** as

$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}},$$

where A_i and B_i are the i th **components** of vectors **A** and **B**, respectively.

The resulting similarity ranges from -1 meaning exactly opposite, to 1 meaning exactly the same, with 0 indicating **orthogonality** or **decorrelation**, while in-between values indicate intermediate similarity or dissimilarity.

5.2.1. K- Nearest Neighbors



- We then assign the data point to the class with the highest score.
- Weighting by similarities is often more accurate than simple voting.
- For example, if two classes have the same number of neighbors in the top k , the class with the more similar neighbors wins.
- The Fig. 5.2.7 summarizes the kNN algorithm.

5.2.1. K- Nearest Neighbors



TRAIN-KNN($\mathbb{C}, \mathbb{D}$)

```
1  $\mathbb{D}' \leftarrow \text{PREPROCESS}(\mathbb{D})$   
2  $k \leftarrow \text{SELECT-K}(\mathbb{C}, \mathbb{D}')$   
3 return  $\mathbb{D}', k$ 
```

APPLY-KNN($\mathbb{C}, \mathbb{D}', k, d$)

```
1  $S_k \leftarrow \text{COMPUTENEARESTNEIGHBORS}(\mathbb{D}', k, d)$   
2 for each  $c_j \in \mathbb{C}$   
3 do  $p_j \leftarrow |S_k \cap c_j|/k$   
4 return  $\arg \max_j p_j$ 
```

- Fig. 5.2.7. kNN training (with preprocessing) and testing. p_j is an estimate for $P(c_j | S_k) = P(c_j | d)$. c_j denotes the set of all data points in the class c_j .

5.2.1. K- Nearest Neighbors



- The python code and output results can be presented as follows:

- `import matplotlib.pyplot as plt`
- `from sklearn.neighbors import KNeighborsClassifier`
- `#Create two arrays (x and y) of data points and target class array (classes):`
- `x = [4, 5, 10, 4, 3, 11, 14 , 8, 10, 12]`
- `y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]`
- `classes = [0, 0, 1, 0, 0, 1, 1, 0, 1, 1]`
- `#Transform the x and y arrays into a set of points:`
- `data = list(zip(x, y))`
- `print(data)`

- Output

```
[(4, 21), (5, 19), (10, 24), (4, 17), (3, 16), (11, 25), (14, 24), (8, 22), (10, 21), (12, 21)]
```

5.2.1. K- Nearest Neighbors



- Based on the data points and target class, we fit KNN method for 1 nearest neighbor:
- ```
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(data, classes)
```

## 5.2.1. K- Nearest Neighbors



Then, we can apply the same KNN object to identify the class of new, unforeseen data point.

First, we design new x and y arrays and then call `knn.predict()` on the new data point to obtain a class of 0 or 1:

```
new_x = 8
new_y = 21
new_point = [(new_x, new_y)]
prediction = knn.predict(new_point)
print(prediction)
```

- Output:
- `[0]`

## 5.2.1. K- Nearest Neighbors

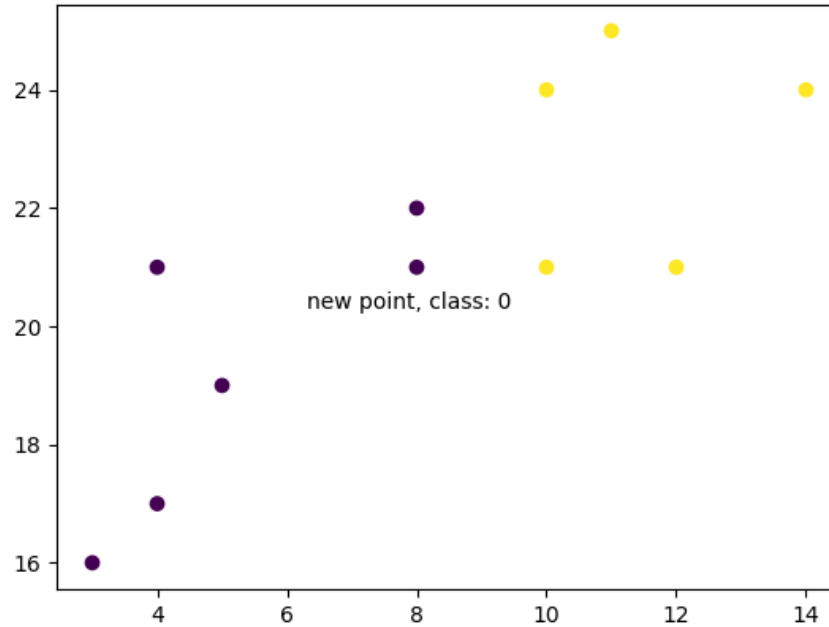


- When we plot all the data along with the new data point and class, we may see it's been labeled blue with the 1 class.
- The text note is to highlight the location of the new point:
- `plt.scatter(x + [new_x], y + [new_y], c=classes + [prediction[0]])`
- `plt.text(x=new_x-1.7, y=new_y-0.7, s=f"new point, class: {prediction[0]}")`
- `plt.show()`

## 5.2.1. K- Nearest Neighbors



- Fig. 5.2.8. Output



## 5.2.1. K- Nearest Neighbors



- However, when we vary the number of neighbors to 5, the number of class used to classify our new data point changes.
- The classification of the new point:
  - `knn = KNeighborsClassifier(n_neighbors=5)`
  - `knn.fit(data, classes)`
  - `prediction = knn.predict(new_point)`
  - `print(prediction)`
- Result:
- `[1]`

## 5.2.1. K- Nearest Neighbors

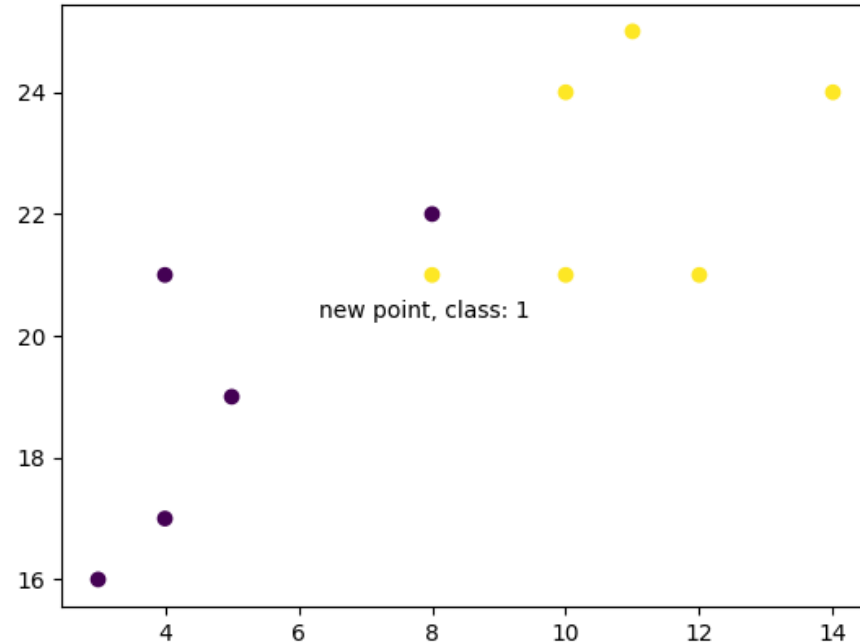


- When we plot the class of the new data point together with the older data points, we can note that the color has changed based on the associated class label:
- `plt.scatter(x + [new_x], y + [new_y], c=classes + [prediction[0]])`
- `plt.text(x=new_x-1.7, y=new_y-0.7, s=f"new point, class: {prediction[0]}")`
- `plt.show()`

## 5.2.1. K- Nearest Neighbors



- Fig. 5.2.9. Output.



## 5.2.1. K- Nearest Neighbors



- **Time complexity and optimality of kNN**
- The Table 5.2.1 gives the time complexity of kNN.
- kNN has properties that are quite different from most other classification algorithms.
- Training a kNN classifier simply consists of determining  $k$  and preprocessing data points.
- In fact, if we preselect a value for  $k$  and do not preprocess, then kNN requires no training at all.

## 5.2.1. K- Nearest Neighbors



- Table 5.2.1. Training and test times for kNN classification.  $M_{ave}$  is the average size of the data points in the input set.

### **kNN with preprocessing of training set**

training  $\Theta(|\mathcal{D}|L_{ave})$

testing  $\Theta(L_a + |\mathcal{D}|M_{ave}M_a) = \Theta(|\mathcal{D}|M_{ave}M_a)$

### **kNN without preprocessing of training set**

training  $\Theta(1)$

testing  $\Theta(L_a + |\mathcal{D}|L_{ave}M_a) = \Theta(|\mathcal{D}|L_{ave}M_a)$

## 5.2.1. K- Nearest Neighbors



- Test time is  $O(|\mathbf{D}|M_{ave}M_a)$  for kNN.
- It is linear in the size of the training set as we need to compute the distance of each training data point from the test data point.
- Test time is independent of the number of classes  $J$ . kNN therefore has a potential advantage for problems with large  $J$ .
- In kNN classification, we do not perform any estimation of parameters as we do in Naive Bayes (priors and conditional probabilities).
- kNN simply memorizes all examples in the training set and then compares the test data point to them.

## 5.2.1. K- Nearest Neighbors



- For this reason, kNN is also called *memory-based learning or instance-based learning*.
- It is usually desirable to have as much training data as possible in machine learning.
- kNN's effectiveness is close to that of the most accurate learning methods of classification.

## 5.2.1. K- Nearest Neighbors



- KNN is a ***nonlinear classifier***.
- The decision boundaries of kNN are locally linear segments, but in general have a complex shape that is not equivalent to a line in 2D or a hyperplane in higher dimensions.
- If a problem is nonlinear and its class boundaries cannot be approximated well with linear hyperplanes, then nonlinear classifiers are often more accurate than linear classifiers.
- If a problem is linear, it is best to use a simpler linear classifier.

## 5.2.1. K- Nearest Neighbors



- **Pros**
- *It is very simple algorithm to understand and interpret.*
- *It is very useful for nonlinear data because there is no assumption about data in this algorithm.*
- *It is a versatile algorithm as we can use it for classification as well as regression.*
- *It has relatively high accuracy but there are much better supervised learning models than K-Nearest Neighbors.*
- *It is robust to the noisy training data*
- *It can be more effective if the training data is large.*

## 5.2.1. K- Nearest Neighbors



- **Cons**
- *Always needs to determine the value of  $K$  which may be complex some time.*
- *The computation cost is high because of calculating the distance between the data points for all the training samples, so it stores all the training data.*
- *High memory storage required as compared to other supervised learning*
- *Operation is slow in case of big  $N$ .*
- *It is very sensitive to the scale of data as well as irrelevant features.*
- *Works on pre-processing stage more before going for KNN like outlier, noise removal.*

## 5.2.1. K- Nearest Neighbors



- The following are some of the areas in which K-Nearest Neighbors can be applied successfully:
- ***Banking System***
- *K-Nearest Neighbors can be used in banking system to predict whether an individual is fit for loan approval.*
- ***Calculating Credit Ratings***
- *K-Nearest Neighbors algorithms can be used to find an individual's credit rating.*

## 5.2.1. K- Nearest Neighbors



- **Politics**
- *With the help of K-Nearest Neighbors algorithms, we can classify a potential voter into various classes like “Will Vote”, “Will not Vote”, “Will Vote to Party ‘Congress’”, “Will Vote to Party ‘BJP’”.*
- *Other areas in which K-Nearest Neighbors algorithm can be used are Speech Recognition, Handwriting Detection, Image Recognition and Video Recognition.*

## 5.2.2. Decision Tree



- ***Decision Tree is a Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems.*
- It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules, and each leaf node represents the outcome.**
- *In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node**.*
- Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.

## 5.2.2. Decision Tree

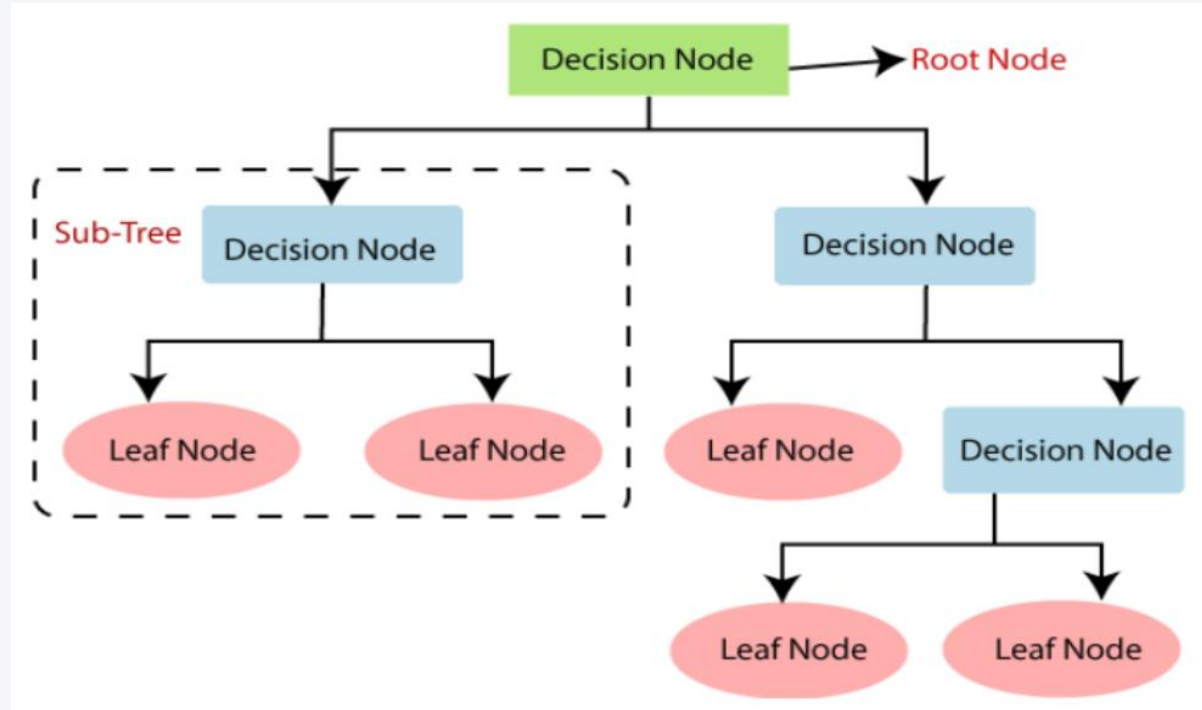


- The decisions or the test are performed on the basis of data/features of the given dataset.
- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- A decision tree simply asks a question and based on the answer (Yes/No), it further split the tree into subtrees (Fig. 5.2.10).
- Below diagram explains the general structure of a decision tree:
- *Note: A decision tree can contain categorical data (YES/NO) as well as numeric data.*

## 5.2.2. Decision Tree



- Fig. 5.2.10.



## 5.2.2. Decision Tree



- ***Below are the two reasons for using the Decision tree:***
- *Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.*
- *The logic behind the decision tree can be easily understood because it shows a tree-like structure.*
- In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree.
- This algorithm compares the values of root attribute with the record (real dataset) attribute and based on the comparison, follows the branch and jumps to the next node.

## 5.2.2. Decision Tree



- For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further.
- It continues the process until it reaches the leaf node of the tree.
- *The complete process can be better understood using the below algorithm:*
- **Step-1:** Begin the tree with the root node, says  $S$ , which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM)**.
- **Step-3:** Divide the  $S$  into subsets that contains possible values for the best attributes.

## 5.2.2. Decision Tree



- **Step-4:** *Generate the decision tree node, which contains the best attribute.*
- **Step-5:** *Recursively make new decision trees using the subsets of the dataset created in step -3.*
- Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

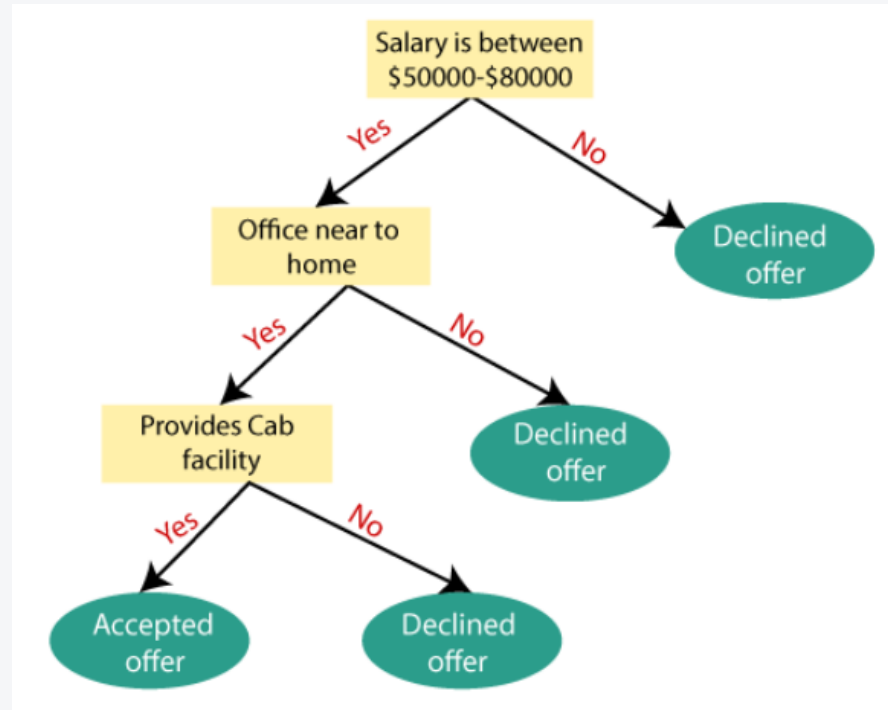
## 5.2.2. Decision Tree



- **Example:** Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not.
- So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM).
- The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels.
- The next decision node further gets split into one decision node (Cab facility) and one leaf node.
- Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer).
- Consider the below diagram (Fig. 5.2.11):

## 5.2.2. Decision Tree

- Fig. 5.2.11.



## 5.2.2. Decision Tree



- The python code and output results can be presented as follows:
- **Step 1:** Import the required libraries
  - `# import numpy package for arrays and stuff`
  - `import numpy as np`
  - 
  - `# import matplotlib.pyplot for plotting our result`
  - `import matplotlib.pyplot as plt`
  - 
  - `# import pandas for importing csv files`
  - `import pandas as pd`
-

## 5.2.2. Decision Tree



- **Step 2:** Initialize and print the Dataset

- `# import dataset`
- `# dataset = pd.read_csv('Data.csv')`
- `# alternatively open up .csv file to read data`
  
- `dataset = np.array(`
- `[['Asset Flip', 100, 1000],`
- `['Text Based', 500, 3000],`
- `['Visual Novel', 1500, 5000],`
- `['2D Pixel Art', 3500, 8000],`
- `['2D Vector Art', 5000, 6500],`
- `['Strategy', 6000, 7000],`
- `])`

## 5.2.2. Decision Tree



- ['First Person Shooter', 8000, 15000],
- ['Simulator', 9500, 20000],
- ['Racing', 12000, 21000],
- ['RPG', 14000, 25000],
- ['Sandbox', 15500, 27000],
- ['Open-World', 16500, 30000],
- ['MMOFPS', 25000, 52000],
- ['MMORPG', 30000, 80000]
- # print the dataset
- print(dataset)

```
[['Asset Flip' '100' '1000']
 ['Text Based' '500' '3000']
 ['Visual Novel' '1500' '5000']
 ['2D Pixel Art' '3500' '8000']
 ['2D Vector Art' '5000' '6500']
 ['Strategy' '6000' '7000']]
```

## 5.2.2. Decision Tree



- **Step 3:** Select all the rows and column 1 from the dataset to “X”.

- *# select all rows by : and column 1*
- *# by 1:2 representing features*
- *X = dataset[:, 1:2].astype(int)*
- 
- *# print X*
- *print(X)*

```
[[100]
 [500]
 [1500]
 [3500]
 [5000]
 [6000]]
```

## 5.2.2. Decision Tree



- **Step 4:** Select all of the rows and column 2 from the dataset to “y”.

- *# select all rows by : and column 2*
- *# by 2 to Y representing labels*
- *y = dataset[:, 2].astype(int)*
- 
- *# print y*
- *print(y)*

```
[1000 3000 5000 8000 6500 7000]
```

## 5.2.2. Decision Tree



- **Step 5:** Fit decision tree regressor to the dataset
- # import the regressor
- from sklearn.tree import DecisionTreeRegressor
- 
- # create a regressor object
- regressor = DecisionTreeRegressor(random\_state = 0)
- 
- # fit the regressor with X and Y data
- regressor.fit(X, y)
- 
- # print the dataset
- print(dataset)

```
[['Asset Flip' '100' '1000']
 ['Text Based' '500' '3000']
 ['Visual Novel' '1500' '5000']
 ['2D Pixel Art' '3500' '8000']
 ['2D Vector Art' '5000' '6500']
 ['Strategy' '6000' '7000']]
```

## 5.2.2. Decision Tree



- **Step 6:** Predicting a new value
- 
- `# predicting a new value`
- `# test the output by changing values, like 3750`
- `y_pred = regressor.predict([[3750]])`
- 
- `# print the predicted price`
- `print("Predicted price: % r\n"% y_pred)`

Predicted price: 8000

## 5.2.2. Decision Tree



- **Step 7: Visualizing the result**
  - # arange for creating a range of values
  - # from min value of X to max value of X
  - # with a difference of 0.01 between two
  - # consecutive values
  - `X_grid = np.arange(min(X), max(X), 0.01)`
  - # reshape for reshaping the data into
  - # a `len(X_grid)*1` array, i.e. to make
  - # a column out of the `X_grid` values
  - `X_grid = X_grid.reshape((len(X_grid), 1))`
  - # scatter plot for original data
  - `plt.scatter(X, y, color = 'red')`

## 5.2.2. Decision Tree

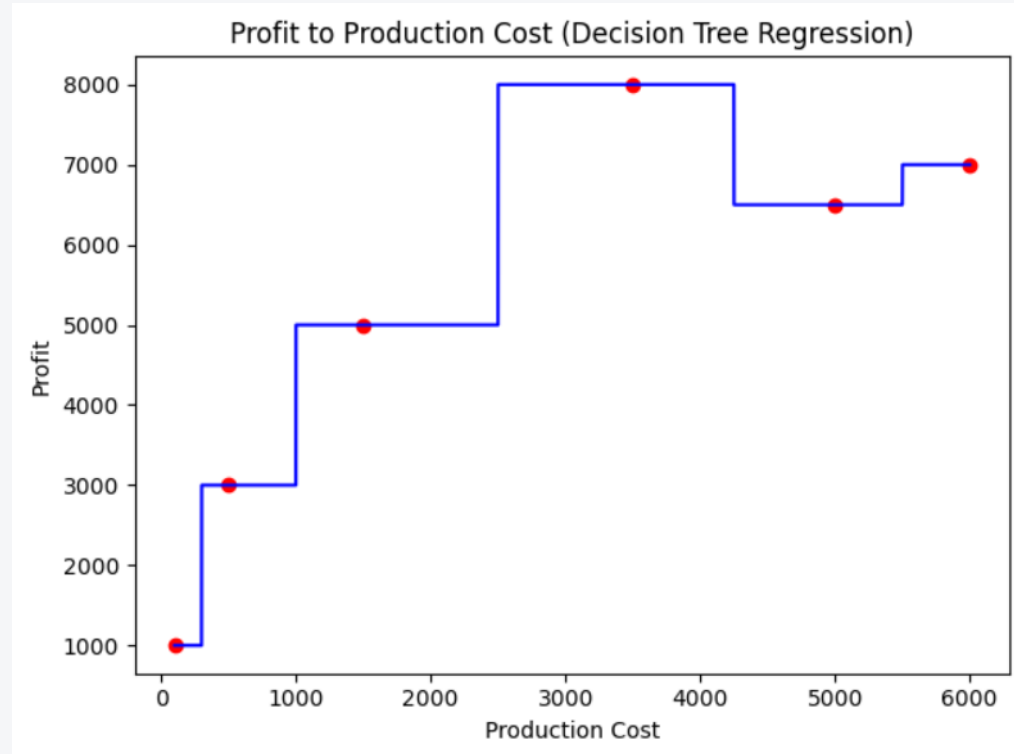


- `# plot predicted data`
- `plt.plot(X_grid, regressor.predict(X_grid), color = 'blue')`
- `# specify title`
- `plt.title('Profit to Production Cost (Decision Tree Regression)')`
- `#specify X axis label`
- `plt.xlabel('Production Cost')`
- `# specify Y axis label`
- `plt.ylabel('Profit')`
- `# show the plot`
- `plt.show()`

## 5.2.2. Decision Tree



- Fig. 5.2.15.2.
- Output



## 5.2.2. Decision Tree



- **Advantages**
- *1. The tree structure can handle both categorical and numerical features in a natural and straightforward way.*
- *Specifically, there is no need to pre-process categorical features, say via the introduction of dummy variables.*
- *5.2. The final tree obtained after the training phase can be compactly stored for the purpose of making predictions for new feature vectors.*
- *The prediction process only involves a single tree traversal from the tree root to a leaf.*

## 5.2.2. Decision Tree



- *3. The hierarchical nature of decision trees allows for an encoding of the feature's conditional information.*
- *4. The tree structure can be easily understood and interpreted by domain experts with little statistical knowledge, since it is essentially a logical decision flow diagram.*
- *5.2. The sequential decision tree growth procedure, and in particular the fact that the tree has been split using the most important features, provides an implicit step-wise variable elimination procedure. In addition, the partition of the variable space into smaller regions results in simpler prediction problems in these regions.*
- *6. Decision trees are invariant under monotone transformations (rank/order preserving property) of the data.*

## 5.2.2. Decision Tree



- *7. In the classification setting, it is common to report not only the predicted value of a feature vector, but also the respective class probabilities. Decision trees handle this task without any additional effort.*
- *8. As each training point is treated equally in the construction of a tree, their structure of the tree will be relatively robust to outliers (far points). In a way, trees exhibit a similar kind of robustness as the sample median does for real-valued data.*

## 5.2.2. Decision Tree



- **Limitations**
- *Despite the fact that the decision trees are extremely interpretable, the predictive accuracy is generally worse to other established statistical learning methods.*
- *In addition, decision trees, and in particular very deep trees that were not subject to pruning, are heavily dependent on their training set.*
- *A small change in the training set can result in a dramatic change of the resulting decision tree.*

## 5.2.2. Decision Tree



- **Applications of Decision Trees**
- *Evaluating prospective growth opportunities for businesses based on historical data.*
- *Use of demographic data to find prospective clients.*
- *The use of a decision tree support tool can help lenders evaluate a customer's creditworthiness to prevent losses.*
- *Decision trees can also be used in operations research in planning logistics and can help in determining appropriate strategies that will help a company achieve its intended goals.*
- *Other fields where decision trees can be applied include engineering, education, law, business, healthcare, and finance.*

## 5.2.3. Linear Regression

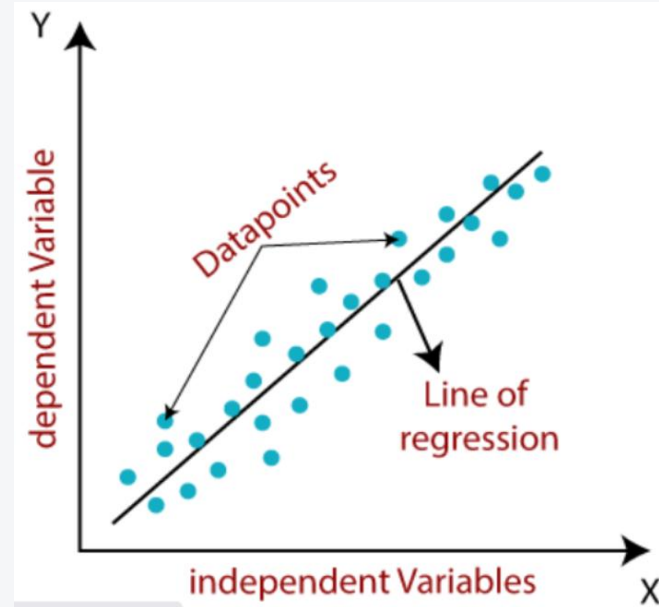


- **Linear regression is one of the easiest and most popular supervised Machine Learning algorithms.**
- It is a statistical method that is used for predictive analysis.
- Linear regression makes predictions for continuous/real or numeric variables such as **sales, salary, age, product price**, etc.
- Linear regression algorithm shows a linear relationship between a dependent ( $y$ ) and one or more independent ( $x$ ) variables, hence called as linear regression.
- Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable.

## 5.2.3. Linear Regression

- The linear regression model provides a sloped straight line representing the relationship between the variables.
- Consider the below image (Fig. 5.2.13):

Fig. 5.2.13.



## 5.2.3. Linear Regression



- Mathematically, we can represent a linear regression as:
- $y = a_0 + a_1x + \epsilon$
- **Here,**
- Y= Dependent Variable (Target Variable)  
X= Independent Variable (predictor Variable)  
a0= intercept of the line (Gives an additional degree of freedom)  
a1 = Linear regression coefficient (scale factor to each input value).  
 $\epsilon$  = random error
- The values for x and y variables are training datasets for Linear Regression model representation.

## 5.2.3. Linear Regression



- **Types of Linear Regression**
- Linear regression can be further divided into two types of the algorithm:
- **Simple Linear Regression:**  
If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.
- **Multiple Linear regression:**  
If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.

## 5.2.3. Linear Regression

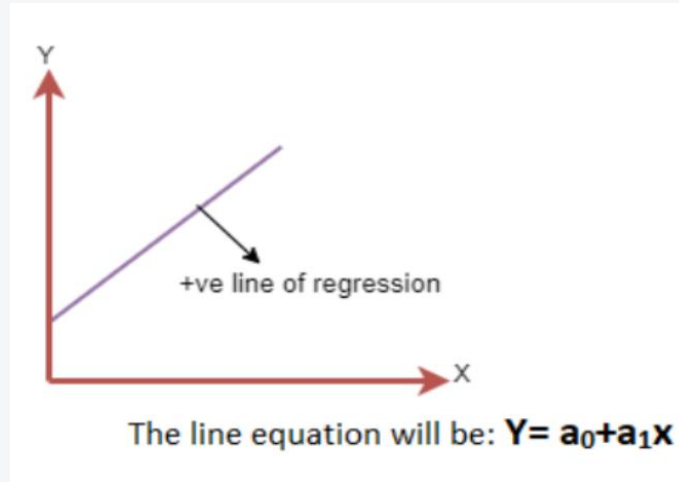


- **Linear Regression Line**
- A linear line showing the relationship between the dependent and independent variables is called a **regression line**.
- A regression line can show two types of relationship:
- **Positive Linear Relationship**  
If the dependent variable increases on the Y-axis and independent variable increases on X-axis, then such a relationship is termed as a Positive linear relationship (Fig. 5.2.14).

## 5.2.3. Linear Regression



- Fig. 5.2.14.



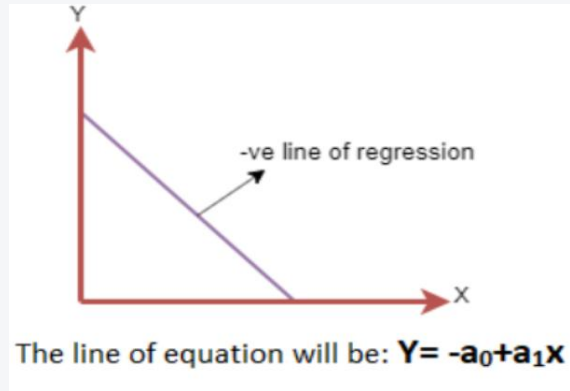
## 5.2.3. Linear Regression



- **Negative Linear Relationship**

If the dependent variable decreases on the Y-axis and independent variable increases on the X-axis, then such a relationship is called a negative linear relationship (Fig. 5.2.15).

- Fig. 5.2.15.



## 5.2.3. Linear Regression



- **Finding the best fit line:**
- When working with linear regression, our main goal is to find the best fit line that means the error between predicted values and actual values should be minimized.
- The best fit line will have the least error.
- The different values for weights or the coefficient of lines ( $a_0, a_1$ ) gives a different line of regression, so we need to calculate the best values for  $a_0$  and  $a_1$  to find the best fit line, so to calculate this we use cost function.

## 5.2.3. Linear Regression



- **Cost function-**
- The different values for weights or coefficient of lines ( $a_0, a_1$ ) gives the different line of regression, and the cost function is used to estimate the values of the coefficient for the best fit line.
- Cost function optimizes the regression coefficients or weights. It measures how a linear regression model is performing.
- We can use the cost function to find the accuracy of the **mapping function**, which maps the input variable to the output variable.
- This mapping function is also known as **Hypothesis function**.

## 5.2.3. Linear Regression



- For Linear Regression, we use the **Mean Squared Error (MSE)** cost function, which is the average of squared error occurred between the predicted values and actual values.

- It can be written as:

$$MSE = \frac{1}{N} \sum_{i=1}^n (y_i - (a_1 x_i + a_0))^2, \quad (5.2.2)$$

- where
- N is total number of observation,  
Y<sub>i</sub> is actual value,  
(a<sub>1</sub>x<sub>i</sub>+a<sub>0</sub>) is predicted value.

## 5.2.3. Linear Regression



- **Residuals:** The distance between the actual value and predicted values is called residual.
- If the observed points are far from the regression line, then the residual will be high, and so cost function will high.
- If the scatter points are close to the regression line, then the residual will be small and hence the cost function.

## 5.2.3. Linear Regression



- **Gradient Descent:**
- Gradient descent is used to minimize the MSE by calculating the gradient of the cost function.
- A regression model uses gradient descent to update the coefficients of the line by reducing the cost function.
- It is done by a random selection of values of coefficient and then iteratively update the values to reach the minimum cost function.

## 5.2.3. Linear Regression



- **Model Performance:**
- The Goodness of fit determines how the line of regression fits the set of observations.
- The process of finding the best model out of various models is called **optimization**.
- It can be achieved by below method:
- **Assumptions of Linear Regression**
- Below are some important assumptions of Linear Regression.
- These are some formal checks while building a Linear Regression model, which ensures to get the best possible result from the given dataset.

## 5.2.3. Linear Regression



- **Linear relationship between the features and target:**  
Linear regression assumes the linear relationship between the dependent and independent variables.
- **Small or no multicollinearity between the features:**  
Multicollinearity means high-correlation between the independent variables.
- Due to multicollinearity, it may difficult to find the true relationship between the predictors and target variables.
- Or we can say, it is difficult to determine which predictor variable is affecting the target variable and which is not.
- So, the model assumes either little or no multicollinearity between the features or independent variables.

## 5.2.3. Linear Regression



- **Homoscedasticity Assumption:**

Homoscedasticity is a situation when the error term is the same for all the values of independent variables.

- With homoscedasticity, there should be no clear pattern distribution of data in the scatter plot.

- **Normal distribution of error terms:**

Linear regression assumes that the error term should follow the normal distribution pattern.

- If error terms are not normally distributed, then confidence intervals will become either too wide or too narrow, which may cause difficulties in finding coefficients.

## 5.2.3. Linear Regression



- **No autocorrelations:**

The linear regression model assumes no autocorrelation in error terms. If there will be any correlation in the error term, then it will drastically reduce the accuracy of the model.

- Autocorrelation usually occurs if there is a dependency between residual errors.
- The sum of squares of differences between the actual output and the predicted output is minimized using a linear function.
- This method is called **Ordinary Least Squares**.
- The main advantage of linear regression is that it is not complex.
- You may also find more accurate models in non-linear regression, but they will be slower.

## 5.2.3. Linear Regression



- **Python Code Example**

- `import numpy as np`
- `import matplotlib.pyplot as plt`
- `from sklearn.linear_model import LinearRegression`
  
- `x = 30 * np.random.random((20, 1))`
  
- `y = 0.5 * x + 1.0 + np.random.normal(size=x.shape)`
  
- `model = LinearRegression()`
- `model.fit(x, y)`

## 5.2.3. Linear Regression

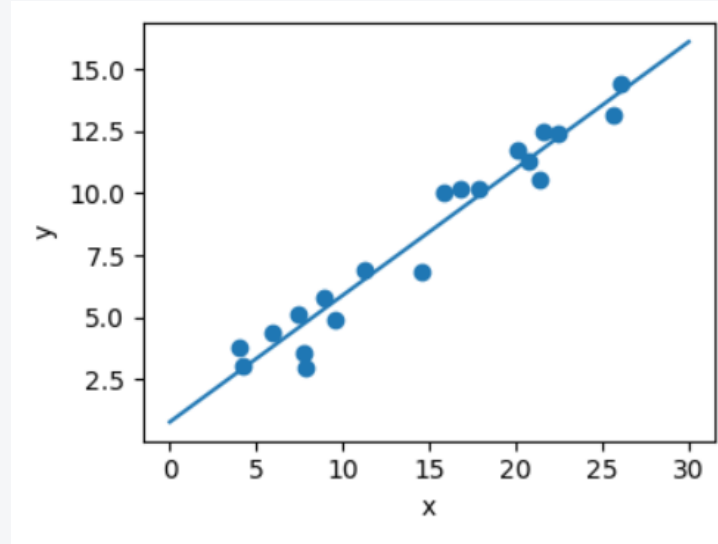


- `x_new = np.linspace(0, 30, 100)`
- `y_new = model.predict(x_new[:, np.newaxis])`
- `plt.figure(figsize=(4, 3))`
- `ax = plt.axes()`
- `ax.scatter(x, y)`
- `ax.plot(x_new, y_new)`
- `ax.set_xlabel('x')`
- `ax.set_ylabel('y')`
- `ax.axis('tight')`
- `plt.show()`

## 5.2.3. Linear Regression



- Fig. 5.2.16.
- Output.



## 5.2.3. Linear Regression



- The straight line can be seen in the plot, showing how linear regression attempts to draw a straight line that will best minimize the residual sum of squares between the observed responses in the dataset, and the responses predicted by the linear approximation.
- **The applications of ML regression algorithms are as follows:**
- **Forecasting or Predictive analysis:** One of the important uses of regression is forecasting or predictive analysis.
- For example, we can forecast GDP, oil prices or in simple words the quantitative data that changes with the passage of time.

## 5.2.3. Linear Regression



- **Optimization:** We can optimize business processes with the help of regression.
- For example, a store manager can create a statistical model to understand the peak time of coming of customers.
- **Error correction:** In business, taking correct decision is equally important as optimizing the business process.
- Regression can help us to take correct decision as well in correcting the already implemented decision.

## 5.2.3. Linear Regression



- **Economics:** It is the most used tool in economics.
- We can use regression to predict supply, demand, consumption, inventory investment etc.
- **Finance:** A financial company is always interested in minimizing the risk portfolio and want to know the factors that affects the customers.
- All these can be predicted with the help of regression model.

## 5.2.3. Linear Regression



- **Advantages**
  - Linear regression performs exceptionally well for linearly separable data
  - Easier to implement, interpret and efficient to train
- **Disadvantages**
  - The assumption of linearity between dependent and independent variables
  - It is often quite prone to noise and overfitting

## 5.2.4. Logistic Regression



- **Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique.**
- It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable.
- Therefore, the outcome must be a categorical or discrete value.
- It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1.**

## 5.2.4. Logistic Regression



- Logistic Regression is much similar to the Linear Regression except that how they are used.
- Linear Regression is used for solving Regression problems, whereas **Logistic regression is used for solving the classification problems.**
- In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).
- The curve from the logistic function indicates the likelihood of something such as whether the cells are cancerous or not, a mouse is obese or not based on its weight, etc.
- Logistic Regression is a significant machine learning algorithm because it has the ability to provide probabilities and classify new data using continuous and discrete datasets.

## 5.2.4. Logistic Regression

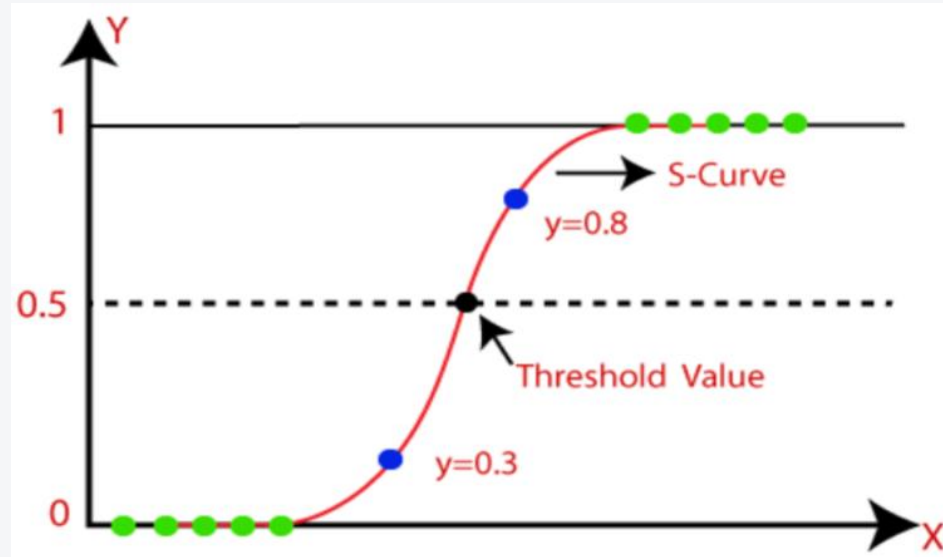


- Logistic regression uses the concept of predictive modeling as regression; therefore, it is called logistic regression, but is used to classify samples;
- Therefore, it falls under ***the classification algorithm.***
- **Logistic Function (Sigmoid Function)**
- Logistic Regression can be used to classify the observations using different types of data and can easily determine the most effective variables used for the classification.
- The below image is showing the logistic function (Fig. 5.2.17):

## 5.2.4. Logistic Regression



- Fig. 5.2.17.



## 5.2.4. Logistic Regression



- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1.
- The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form.
- The S-form curve is called the logistic function or the Sigmoid function.
- In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1.
- Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.

## 5.2.4. Logistic Regression



- It is used to estimate discrete values or values like 0/1, Y/N, T/F based on the given set of independent variable(s).
- It predicts the probability of occurrence of an event by fitting data to a logit function.
- Hence, it is also called **logit regression**.
- Since, it predicts the probability, its output values lie between 0 and 1.
- It is an algorithm that comes from statistics and is used for supervised classification problems.

## 5.2.4. Logistic Regression



- **Logistic Regression Equation**

- In logistic regression, we seek to find the vector  $\beta$  of parameters in the following equation that minimize the cost function.

- $$\text{logit}(p_i) = \ln \left( \frac{p_i}{1 - p_i} \right) = \beta_0 + \beta_1 x_{1,i} + \dots + \beta_k x_{k,i} \quad (5.2.3)$$

- **Type of Logistic Regression**

- In general case, on the basis of the categories, Logistic Regression can be classified into three types:
- **Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.

## 5.2.4. Logistic Regression



- **Multinomial:** In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep"
- **Ordinal:** In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High".
- In order to understand this algorithm, consider a simple example.
- Assume that there is a puzzle to solve that has only 2 outcome scenarios – either there is a solution or there is none.
- Now suppose, we have a wide range of puzzles to test a person which subjects he is good at.

## 5.2.4. Logistic Regression



- The outcomes may be something like this – if a trigonometry puzzle is given, a person may be 80% likely to solve it.
- On the other hand, if a geography puzzle is given, the person may be only 20% likely to solve it.
- This is where Logistic Regression helps in solving.
- As per math, the log odds of the outcome is expressed as a linear combination of the predictor variables.
- $\text{odds} = p / (1-p)$  = probability of event occurrence / probability of not event occurrence
- $\ln(\text{odds}) = \ln(p/(1-p))$  ;  $\ln$  is the logarithm to the base 'e'.
- $\text{logit}(p) = \ln(p/(1-p)) = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_kx_k$

## 5.2.4. Logistic Regression



- Note that in the above  $p$  is the probability of presence of the characteristic of interest.
- It chooses parameters that maximize the likelihood of observing the sample values rather than that minimize the sum of squared errors (like in ordinary regression).
- The following points may be note-worthy when working on logistic regression –
- It is similar to regression in that the objective is to find the values for the coefficients that weight each input variable.

## 5.2.4. Logistic Regression



- Unlike in linear regression, the prediction for the output is found using a non-linear function called the logistic function.
- The logistic function appears like a big 'S' and will change any value into the range 0 to 1.
- This is useful because we can apply a rule to the output of the logistic function to assign values to 0 and 1 and predict a class value.
- The following code shows how to develop a plot for logistic expression where a synthetic dataset is classified into values as either 0 or 1, that is class one or two, using the logistic curve.
- The python code and output plot of Logistic Regression model can be presented as follows:

## 5.2.4. Logistic Regression



- `import numpy as np`
- `import matplotlib.pyplot as plt`
- `from sklearn import linear_model`
- `# This is the test set, it's a straight line with some Gaussian noise`
- `xmin, xmax = -10, 10`
- `n_samples = 100`
- `np.random.seed(0)`
- `X = np.random.normal(size = n_samples)`
- `y = (X > 0).astype(np.float)`

## 5.2.4. Logistic Regression



- `X[X > 0] *= 4`
- `X += .3 * np.random.normal(size = n_samples)`
- `X = X[:, np.newaxis]`
- `# run the classifier`
- `clf = linear_model.LogisticRegression(C=1e5)`
- `clf.fit(X, y)`
- `# and plot the result`
- `plt.figure(1, figsize = (4, 3))`
- `plt.clf()`

## 5.2.4. Logistic Regression



- `plt.scatter(X.ravel(), y, color='black', zorder=20)`
- `X_test = np.linspace(-10, 10, 300)`
- `def model(x):`
- `return 1 / (1 + np.exp(-x))`
- `loss = model(X_test * clf.coef_ + clf.intercept_).ravel()`
- `plt.plot(X_test, loss, color='blue', linewidth=3)`
- `ols = linear_model.LinearRegression()`
- `ols.fit(X, y)`
- `plt.plot(X_test, ols.coef_ * X_test + ols.intercept_, linewidth=1)`

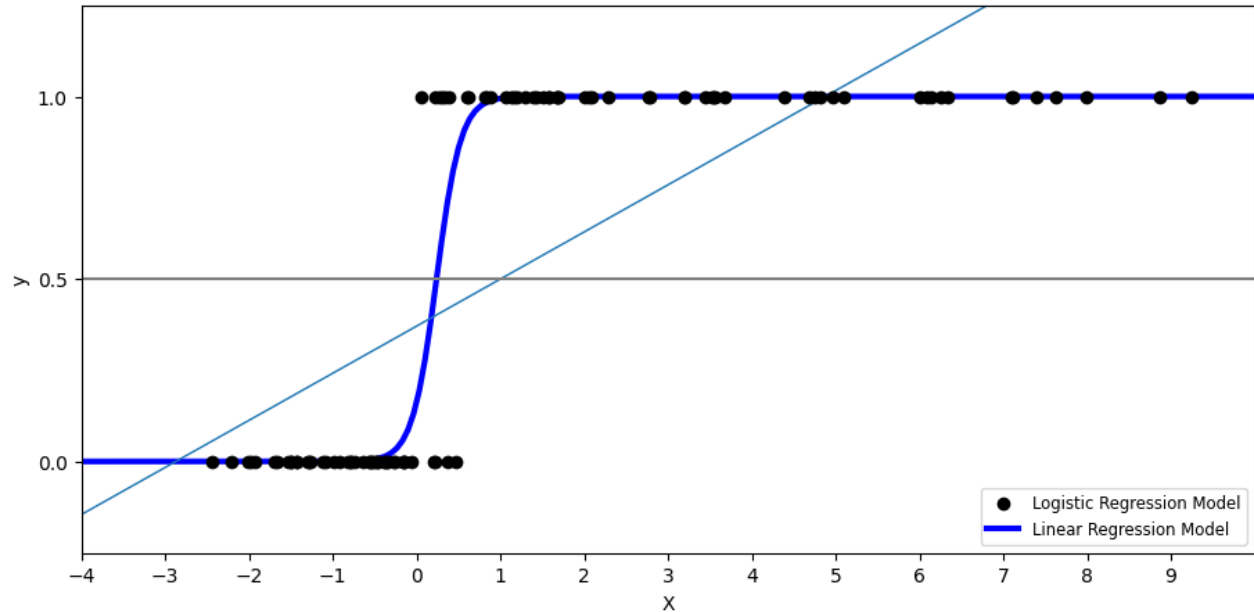
## 5.2.4. Logistic Regression



- `plt.axhline(.5, color='.5')`
- `plt.ylabel('y')`
- `plt.xlabel('X')`
- `plt.xticks(range(-10, 10))`
- `plt.yticks([0, 0.5, 1])`
- `plt.ylim(-.25, 1.25)`
- `plt.xlim(-4, 10)`
- `plt.legend(('Logistic Regression Model', 'Linear Regression Model'),`
- `loc="lower right", fontsize='small')`
- `plt.show()`

## 5.2.4. Logistic Regression

- Fig. 5.2.18.



## 5.2.4. Logistic Regression



- **Pros of Logistic Regression**
- *Logistic regression is easier to implement, interpret, and very efficient to train.*
- *It makes no assumptions about distributions of classes in feature space.*
- *It can easily extend to multiple classes(multinomial regression) and a natural probabilistic view of class predictions.*
- *It not only provides a measure of how appropriate a predictor(coefficient size)is, but also its direction of association (positive or negative).*
- *It is very fast at classifying unknown records.*
- *Good accuracy for many simple data sets and it performs well when the dataset is linearly separable.*
- *It can interpret model coefficients as indicators of feature importance.*

## 5.2.4. Logistic Regression



- **Cons of Logistic Regression**
- *If the number of observations is lesser than the number of features, Logistic Regression should not be used, otherwise, it may lead to overfitting.*
- *The major limitation of Logistic Regression is the assumption of linearity between the dependent variable and the independent variables.*
- *It can only be used to predict discrete functions.*
- *Hence, the dependent variable of Logistic Regression is bound to the discrete number set.*
- *Non-linear problems can't be solved with logistic regression because it has a linear decision surface.*
- *Linearly separable data is rarely found in real-world scenarios.*

## 5.2.4. Logistic Regression



- *Logistic Regression requires average or no multicollinearity between independent variables.*
- *It is hard to obtain complex relationships using logistic regression. More powerful and compact algorithms such as Neural Networks can easily outperform this algorithm.*
- *In Linear Regression independent and dependent variables are related linearly.*
- *But Logistic Regression needs that independent variables are linearly related to the log odds ( $\log(p/(1-p))$ ).*

## 5.2.4. Logistic Regression



- **Applications**
- *Logistic regression is used in various fields, including*
- *machine learning,*
- *most medical fields,*
- *and social sciences.*

## 5.2.5. Support Vector Machines



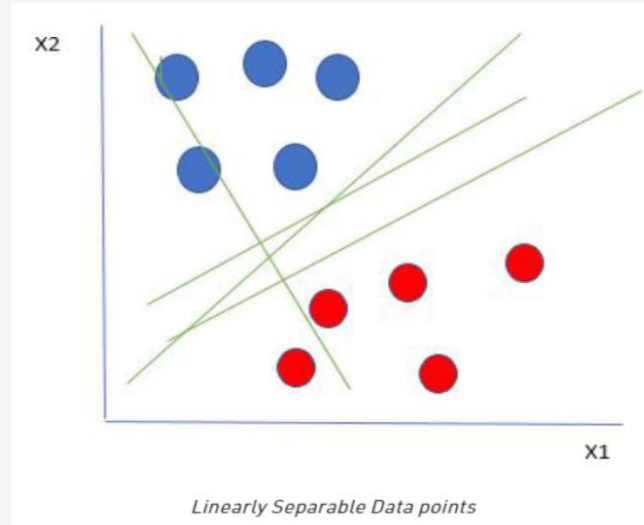
- **Support Vector Machine or SVM is a supervised machine learning algorithm** used for both classification and regression.
- Though we say regression problems as well its ***best suited for classification***.
- The objective of SVM algorithm is to find a hyperplane in an N-dimensional space that distinctly classifies the data points.
- The dimension of the hyperplane depends upon the number of features.
- ***If the number of input features is two, then the hyperplane is just a line.***
- ***If the number of input features is three, then the hyperplane becomes a 2-D plane.***
- It becomes difficult to imagine when the number of features exceeds three.

## 5.2.5. Support Vector Machines



- Let's consider two independent variables  $x_1$ ,  $x_2$  and one dependent variable which is either a blue circle or a red circle (Fig. 5.2.19).

- Fig. 5.2.19.



## 5.2.5. Support Vector Machines

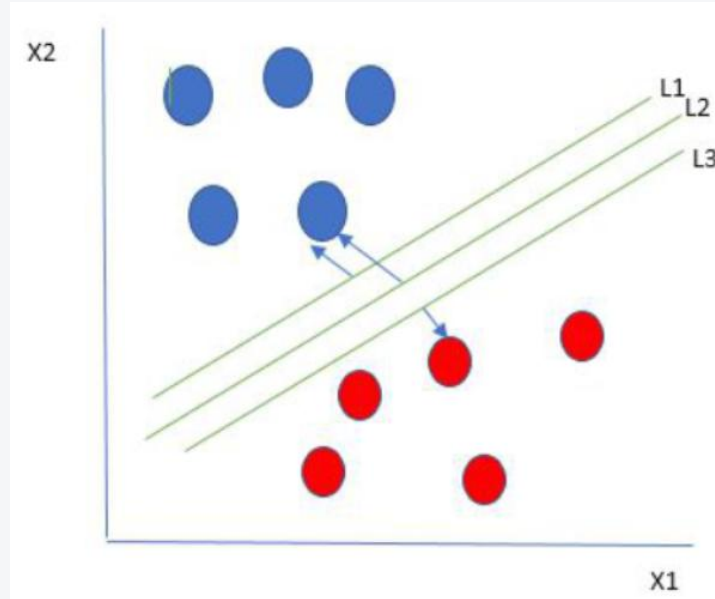


- From the figure above it is very clear that there are multiple lines (our hyperplane here is a line because we are considering only two input features  $x_1$ ,  $x_2$ ) that segregates our data points or does a classification between red and blue circles.
- ***So how do we choose the best line or in general the best hyperplane that segregates our data points?***
- **Selecting the best hyper-plane:**
- ***One reasonable choice as the best hyperplane is the one that represents the largest separation or margin between the two classes*** (Fig. 5.2.20).

## 5.2.5. Support Vector Machines



- Fig. 5.2.20.



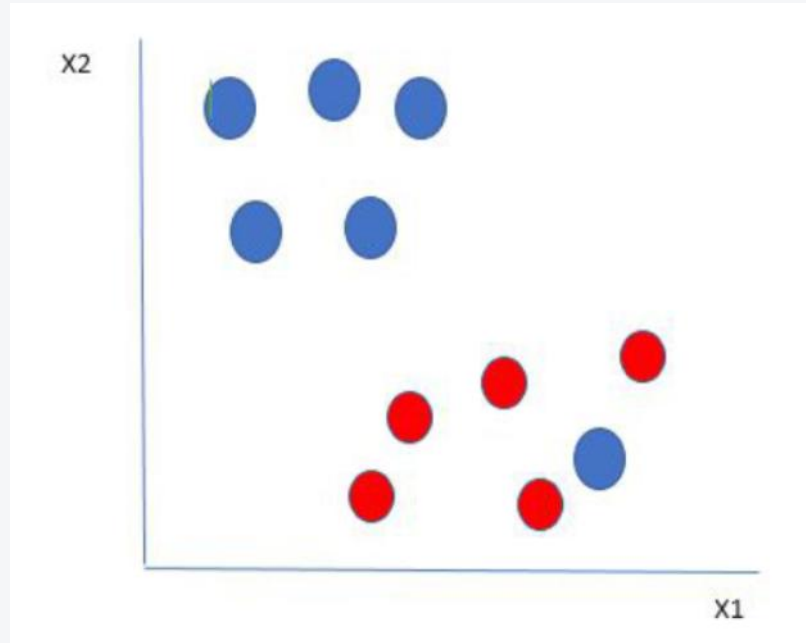
## A close-up, high-angle shot of a person's hands typing on a silver laptop keyboard. The person is wearing a light-colored, long-sleeved shirt with thin vertical stripes. The laptop is open, and the keyboard is black with silver keys. The background is a plain, light-colored surface.

- [3] Simon Haykin, *Neural Networks and Learning Machines*, 3<sup>rd</sup> Edition, Pearson Prentice Hall, 2009.

## 5.2.5. Support Vector Machines



- Fig. 5.2.21.



## 5.2.5. Support Vector Machines

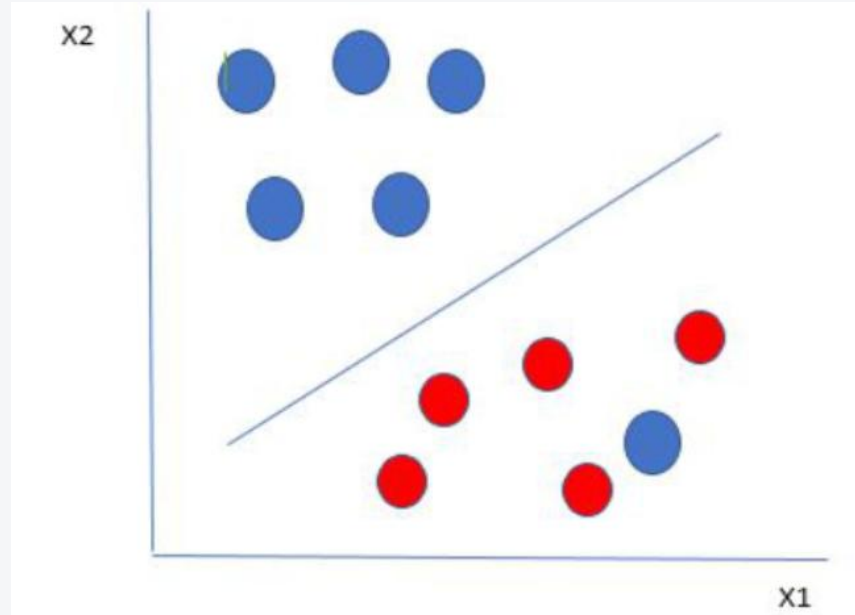


- Here we have one blue ball in the boundary of the red ball.
- So how does SVM classify the data? It's simple!
- The blue ball in the boundary of red ones is an outlier of blue balls.
- The SVM algorithm has the characteristics to ignore the outlier and finds the best hyperplane that maximizes the margin.
- SVM is robust to outliers (Fig. 5.2.22).

## 5.2.5. Support Vector Machines



- Fig. 5.2.25.2.



## 5.2.5. Support Vector Machines



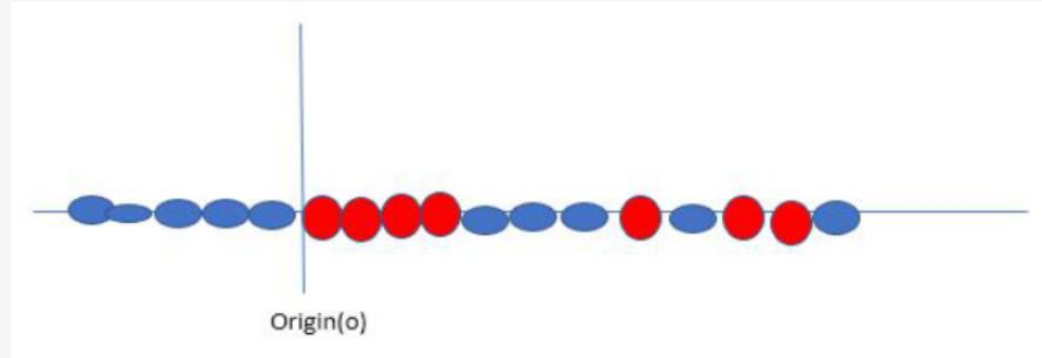
- *So, in this type of data points what SVM does is, it finds maximum margin as done with previous data sets along with that it adds a penalty each time a point crosses the margin.*
- ***So, the margins in these type of cases are called soft margin.***
- ***When there is a soft margin to the data set, the SVM tries to minimize  $(1/\text{margin} + \lambda(\sum \text{penalty}))$ .***
- Hinge loss function is a commonly used penalty. For an intended output  $t = \pm 1$  and a classifier score  $y$ , the hinge loss of the prediction  $y$  is defined as  **$L(y) = \max(1, 1 - ty)$** , where  $y$  is a classifier's decision function.
- *If no violations no hinge loss. If violations are available, hinge loss proportional to the distance of violation.*

## 5.2.5. Support Vector Machines



- Till now, we were talking about linearly separable data(the group of blue balls and red balls are separable by a straight line/linear line).
- *What to do if data are not linearly separable (Fig. 5.2.23)?*

Fig. 5.2.23.

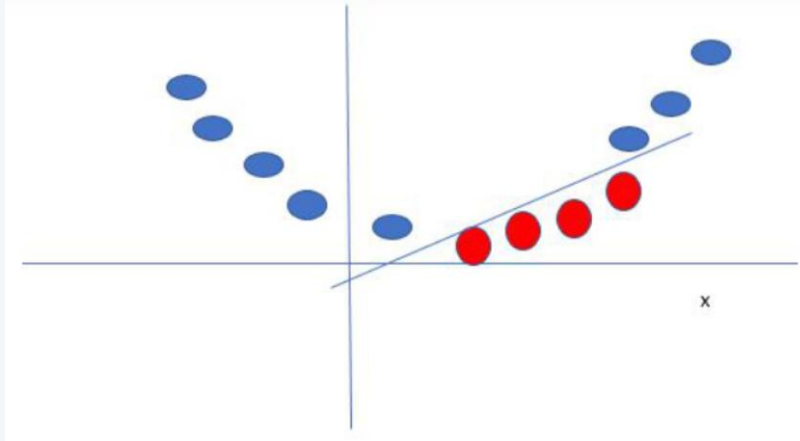


## 5.2.5. Support Vector Machines



- Say, our data is like shown in the figure above. SVM solves this by creating a new variable using a kernel.
- We call a point  $x_i$  on the line, and we create a new variable  $y_i$  as a function of distance from origin  $o$ . so if we plot it we get something like as shown below (Fig. 5.2.24)

Fig. 5.2.24.



## 5.2.5. Support Vector Machines



- In this case, the new variable  $\gamma$  is created as a function of distance from the origin.
- A non-linear function that creates a new variable is referred to as kernel.
- **SVM Kernel:**
  - The SVM kernel is a function that takes low dimensional input space and transforms it into higher-dimensional space, i.e., it converts not separable problem to separable problem.
  - It is mostly useful in non-linear separation problems.
  - Simply put the kernel, it does some extremely complex data transformations then finds out the process to separate the data based on the labels or outputs defined.

## 5.2.5. Support Vector Machines



- Basically, the support vector machine is a binary learning machine with some highly elegant properties.
- To explain how the machine works, it is perhaps easiest to start with the case of separable patterns that arise in the context of pattern classification.
- In this context, the main idea behind the machine may be summed up as follows:
- *Given a training sample, the support vector machine constructs a hyperplane as the decision surface in such a way that the margin of separation between positive and negative examples is maximized.*
- This basic idea is extended in a principled way to deal with the more difficult case of nonlinearly separable patterns.

## 5.2.5. Support Vector Machines



- A notion that is central to the development of the support vector learning algorithm is the *inner-product kernel* between a “support vector”  $\mathbf{x}_i$  and a vector  $\mathbf{x}$  drawn from the input data space.
- Most importantly, the support vectors consist of a small subset of data points extracted by the learning algorithm from the training sample itself.
- Indeed, it is because of this central property that the learning algorithm, involved in the construction of a support vector machine, is also referred to as a *kernel method*.

## 5.2.5. Support Vector Machines



- **OPTIMAL HYPERPLANE FOR LINEARLY SEPARABLE PATTERNS**
- Consider the training sample  $\{(\mathbf{x}_i, d_i)\}_{i=1}^N$ , where  $\mathbf{x}_i$  is the input pattern for the  $i$ th example and  $d_i$  is the corresponding desired response (target output).
- To begin with, we assume that the pattern (class) represented by the subset  $d_i = 1$  and the pattern represented by the subset  $d_i = -1$  are “linearly separable.”
- The equation of a decision surface in the form of a hyperplane that does the separation is
- $$\mathbf{w}^T \mathbf{x} + b = 0 \quad , \quad (5.2.4)$$
- where  $\mathbf{x}$  is an input vector,  $\mathbf{w}$  is an adjustable weight vector, and  $b$  is a bias.
- We may thus write

## 5.2.5. Support Vector Machines



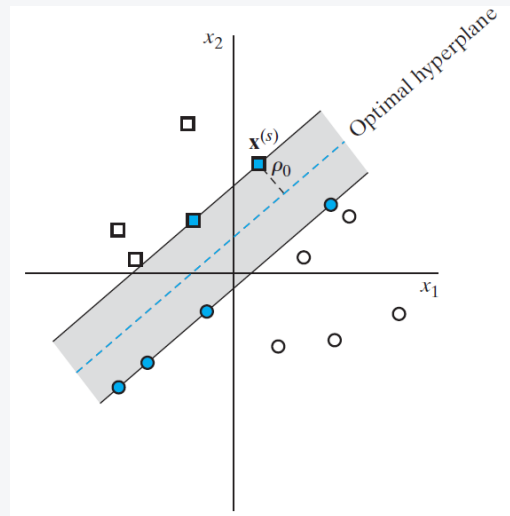
$$\begin{aligned} \mathbf{w}^T \mathbf{x}_i + b &\geq 0 & \text{for } d_i = +1 \\ \mathbf{w}^T \mathbf{x}_i + b &< 0 & \text{for } d_i = -1 \end{aligned} \quad (5.2.5)$$

- The assumption of linearly separable patterns is made here to explain the basic idea behind a support vector machine in a rather simple setting.
- **For a given weight vector  $\mathbf{w}$  and bias  $b$ , the separation between the hyperplane defined in Eq. (5.2.4) and the closest data point is called the *margin of separation*, denoted by  $\gamma$ .**
- *The goal of a support vector machine is to find the particular hyperplane for which the margin of the separation  $\gamma$  is maximized.*
- *Under this condition, the decision surface is*

## 5.2.5. Support Vector Machines



- referred to as the *optimal hyperplane*.
- Fig. 5.2.25 illustrates the geometric construction of an optimal hyperplane for a two-dimensional input space.
- Fig. 5.2.25. Illustration of the idea of
- an optimal hyperplane for linearly separable
- patterns:
- *The data points shaded in blue are support vectors.*



## 5.2.5. Support Vector Machines



- Let  $\mathbf{w}_o$  and  $b_o$  denote the optimum values of the weight vector and bias, respectively.
- Correspondingly, the *optimal hyperplane*, representing a multidimensional linear decision surface in the input space, is defined by

$$\mathbf{w}_o^T \mathbf{x} + b_o = 0 \quad (5.2.6)$$

- which is a rewrite of Eq. (5.2.4).
- The discriminant function

## 5.2.5. Support Vector Machines



- $$g(\mathbf{x}) = \mathbf{w}_o^T \mathbf{x} + b_o \quad (5.2.7)$$
- gives an algebraic measure of the *distance* from  $\mathbf{x}$  to the optimal hyperplane.
- Perhaps the easiest way to see this is to express  $\mathbf{x}$  as

- $$\mathbf{x} = \mathbf{x}_p + r \frac{\mathbf{w}_o}{\|\mathbf{w}_o\|} \quad (5.2.8)$$

-

## 5.2.5. Support Vector Machines



- where  $\mathbf{x}p$  is the normal projection of  $\mathbf{x}$  onto the optimal hyperplane and  $r$  is the desired algebraic distance;  $r$  is positive if  $\mathbf{x}$  is on the positive side of the optimal hyperplane and negative if  $\mathbf{x}$  is on the negative side.
- Since, by definition,  $g(\mathbf{x}p)=0$ , it follows that

$$g(\mathbf{x}) = \mathbf{w}_o^T \mathbf{x} + b_o = r \|\mathbf{w}_o\|$$

- or, equivalently,

$$r = \frac{g(\mathbf{x})}{\|\mathbf{w}_o\|} \quad (5.2.9)$$

## 5.2.5. Support Vector Machines

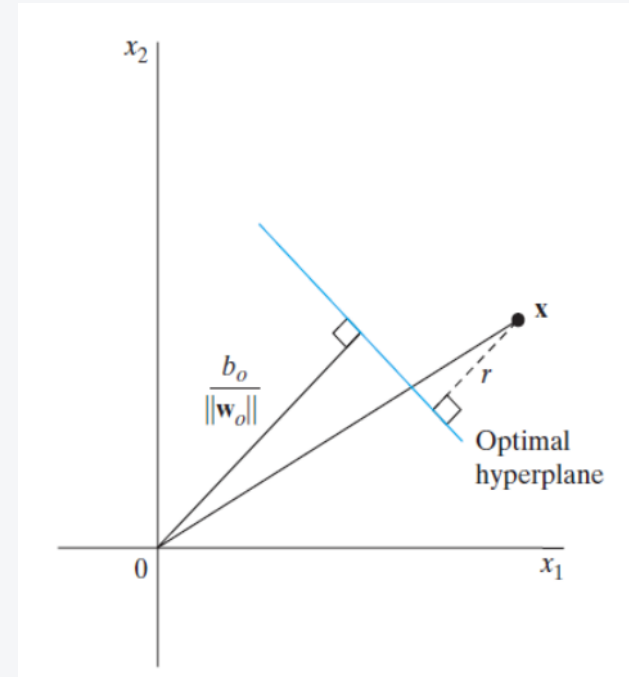


- In particular, the distance from the origin (i.e.,  $\mathbf{x}=\mathbf{0}$ ) to the optimal hyperplane is given by  $b_0/w_0$ .
- If  $b_0 > 0$ , the origin is on the positive side of the optimal hyperplane;
- if  $b_0 < 0$ , it is on the negative side.
- If  $b_0 = 0$ , the optimal hyperplane passes through the origin.
- A geometric interpretation of these algebraic results is given in Fig. 5.2.26.

## 5.2.5. Support Vector Machines



- Fig. 5.2.26. Geometric interpretation distances of points to the two-dimensional case.



geometric  
interpretation

## 5.2.5. Support Vector Machines



- The issue at hand is to find the parameters  $\mathbf{w}_o$  and  $b_o$  for the optimal hyperplane, given the training set  $\mathcal{T} = \{(\mathbf{x}_i, d_i)\}$ .
- In light of the results shown in Fig. 5.2.26, we see that the pair  $(\mathbf{w}_o, b_o)$  must satisfy the following constraint:

$$\begin{aligned} \mathbf{w}_o^T \mathbf{x}_i + b_o &\geq 1 && \text{for } d_i = +1 \\ \mathbf{w}_o^T \mathbf{x}_i + b_o &\leq -1 && \text{for } d_i = -1 \end{aligned} \tag{5.2.10}$$

- Note that if Eq. (5.2.6) holds—that is, if the patterns are linearly separable—we can
- always rescale  $\mathbf{w}_o$  and  $b_o$  such that Eq. (5.2.10) holds; this scaling operation leaves Eq. (5.2.7) unaffected.

## 5.2.5. Support Vector Machines



- *The particular data points  $(\mathbf{x}_i, d_i)$  for which the first or second line of Eq. (5.2.10) is satisfied with the equality sign are called support vectors—hence the name “support vector machine”.*
- All the remaining examples in the training sample are completely irrelevant.
- Because of their distinct property, the support vectors play a prominent role in the operation of this class of learning machines.
- In conceptual terms, the support vectors are those data points that lie closest to the optimal hyperplane and are therefore the most difficult to classify.
- As such, they have a direct bearing on the optimum location of the decision surface.

## 5.2.5. Support Vector Machines



- Consider a support vector  $\mathbf{x}^*(s)$  for which  $d^*(s)=1$ .
- Then, by definition, we have

- $$g(\mathbf{x}^{(s)}) = \mathbf{w}_o^T \mathbf{x}^{(s)} + b_o = \mp 1 \quad \text{for } d^{(s)} = \mp 1 \quad (5.2.11)$$

- From Eq. (5.2.9), the *algebraic distance* from the support vector  $\mathbf{x}(s)$  to the optimal hyperplane is

## 5.2.5. Support Vector Machines



$$\begin{aligned} r &= \frac{g(\mathbf{x}^{(s)})}{\|\mathbf{w}_o\|} \\ &= \begin{cases} \frac{1}{\|\mathbf{w}_o\|} & \text{if } d^{(s)} = +1 \\ -\frac{1}{\|\mathbf{w}_o\|} & \text{if } d^{(s)} = -1 \end{cases} \end{aligned} \quad (5.2.12)$$

- where the plus sign indicates that  $\mathbf{x}^{(s)}$  lies on the positive side of the optimal hyperplane and the minus sign indicates that  $\mathbf{x}^{(s)}$  lies on the negative side of the optimal hyperplane.

## 5.2.5. Support Vector Machines



- *Let  $\rho$  denotes the optimum value of the **margin of separation** between the two classes that constitute the training sample.*
- Then, from Eq. (5.2.12), it follows that

$$\begin{aligned}\rho &= 2r \\ &= \frac{2}{\|\mathbf{w}_o\|}\end{aligned}\tag{5.2.13}$$

- Equation (5.2.13) states the following:
- *Maximizing the margin of separation between binary classes is equivalent to minimizing the Euclidean norm of the weight vector  $\mathbf{w}$ .*

## 5.2.5. Support Vector Machines



- In summary, the optimal hyperplane defined by Eq. (5.2.7) is *unique* in the sense that the optimum weight vector  $\mathbf{w}_0$  provides the maximum possible separation between positive and negative examples.
- This optimum condition is obtained by minimizing the Euclidean norm of the weight vector  $\mathbf{w}$ .

## 5.2.5. Support Vector Machines



- The python code and output plots can be presented as follows:
- `import random`
- `import numpy as np`
- `import pandas as pd`
- `import matplotlib.pyplot as plt`
- `def generate_random_dataset(size):`

## 5.2.5. Support Vector Machines



- #Generate a random dataset and that follows a quadratic distribution
- `x = []`
- `y = []`
- `target = []`
- `for i in range(size):`

## 5.2.5. Support Vector Machines



- `# class zero`
- `x.append(np.round(random.uniform(0, 5.2.5), 1))`
- `y.append(np.round(random.uniform(0, 20), 1))`
- `target.append(0)`
- `# class one`
- `x.append(np.round(random.uniform(1, 5), 2))`
- `y.append(np.round(random.uniform(20, 25), 2))`
- `target.append(1)`
- `x.append(np.round(random.uniform(3, 5), 2))`
- `y.append(np.round(random.uniform(5, 25), 2))`
- `target.append(1)`

## 5.2.5. Support Vector Machines



- # Pandas DataFrame
- `df_x = pd.DataFrame(data=x)`
- `df_y = pd.DataFrame(data=y)`
- `df_target = pd.DataFrame(data=target)`
- `data_frame = pd.concat([df_x, df_y], ignore_index=True, axis=1)`
- `data_frame = pd.concat([data_frame, df_target], ignore_index=True, axis=1)`
- `data_frame.columns = ['x', 'y', 'target']`
- `return data_frame`

## 5.2.5. Support Vector Machines



- # Generate dataset
- size = 100
- dataset = generate\_random\_dataset(size)
- features = dataset[['x', 'y']]
- label = dataset['target']
- # Hold out 20% of the dataset for training
- test\_size = int(np.round(size \* 0.2, 0))
- # Split dataset into training and testing sets
- x\_train = features[:-test\_size].values
- y\_train = label[:-test\_size].values
- x\_test = features[-test\_size:].values
- y\_test = label[-test\_size:].values

## 5.2.5. Support Vector Machines



- `# Plotting the training set`
- `fig, ax = plt.subplots(figsize=(12, 7))`
- `# removing top and right border`
- `ax.spines['top'].set_visible(False)`
- `ax.spines['left'].set_visible(False)`
- `ax.spines['right'].set_visible(False)`
- `# adding major gridlines`
- `ax.grid(color='grey', linestyle='-', linewidth=0.25, alpha=0.5)`
- `ax.scatter(features[: -test_size]['x'], features[: -test_size]['y'], color="#8C7298")`

## 5.2.5. Support Vector Machines



- `from sklearn import svm`
- `model = svm.SVC(kernel='poly', degree=2)`
- `model.fit(x_train, y_train)`
- `plt.show()`
- `fig, ax = plt.subplots(figsize=(12, 7))`
- `# Removing top and right border`
- `ax.spines['top'].set_visible(False)`
- `ax.spines['left'].set_visible(False)`
- `ax.spines['right'].set_visible(False)`

## 5.2.5. Support Vector Machines



- `# Create grid to evaluate model`
- `xx = np.linspace(-1, max(features['x']) + 1, len(x_train))`
- `yy = np.linspace(0, max(features['y']) + 1, len(y_train))`
- `YY, XX = np.meshgrid(yy, xx)`
- `xy = np.vstack([XX.ravel(), YY.ravel()]).T`
- `train_size = len(features[:-test_size]['x'])`
- `# Assigning different colors to the classes`
- `colors = y_train`
- `colors = np.where(colors == 1, '#8C7298', '#4786D1')`
- `# Plot the dataset`
- `ax.scatter(features[:-test_size]['x'], features[:-test_size]['y'], c=colors)`

## 5.2.5. Support Vector Machines



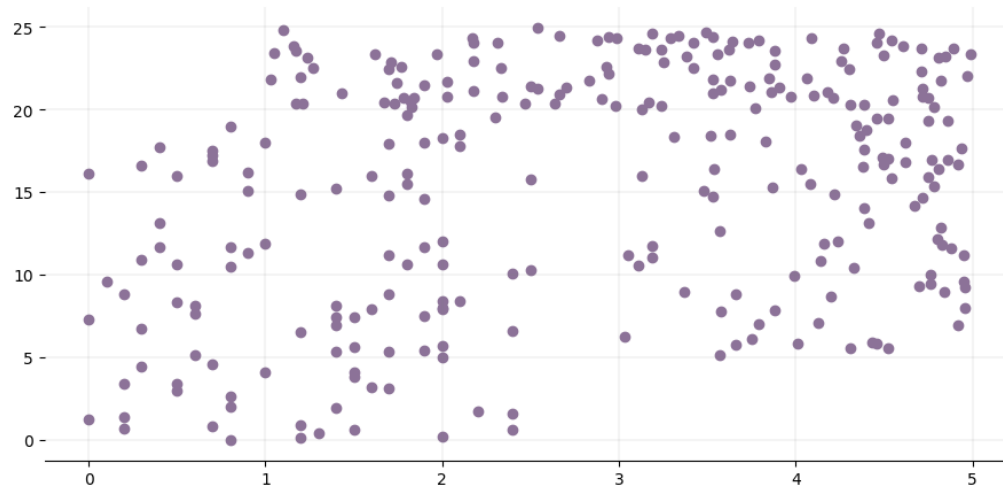
- `# Get the separating hyperplane`
- `Z = model.decision_function(xy).reshape(XX.shape)`
- `# Draw the decision boundary and margins`
- `ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.5, linestyles=['--', '-', '--'])`
- `# Highlight support vectors with a circle around them`
- `ax.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1], s=100, linewidth=1, facecolors='none', edgecolors='k')`
- `plt.show()`
  
- `from sklearn.metrics import accuracy_score`
- `predictions_poly = model.predict(x_test)`
- `accuracy_poly = accuracy_score(y_test, predictions_poly)`
- `print("2nd degree polynomial Kernel\nAccuracy (normalized): " + str(accuracy_poly))`

## 5.2.5. Support Vector Machines



- 2nd degree polynomial Kernel
- Accuracy (normalized): 0.95

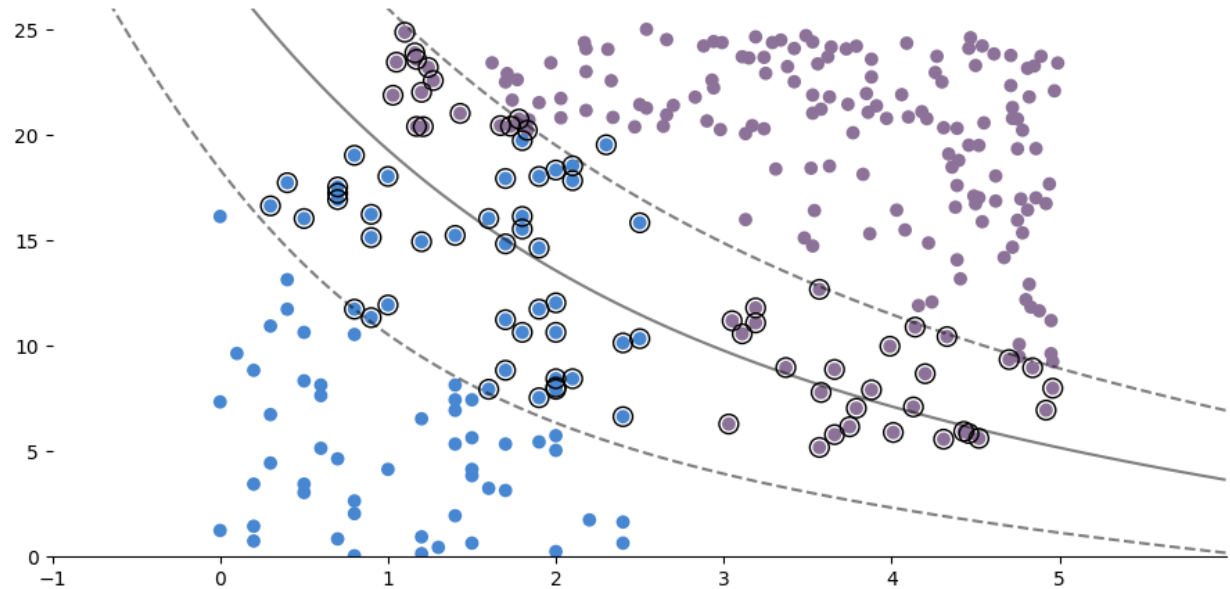
- Fig. 5.2.27.



## 5.2.5. Support Vector Machines



- Fig. 5.2.28.



## 5.2.5. Support Vector Machines



- **Pros of SVM classifiers**
- *SVM classifiers provides **high accuracy** and work well with high dimensional space.*
- *SVM classifiers basically use a subset of training points hence in result uses **very less memory**.*
- **Cons of SVM classifiers**
- *They have **high training time** hence in practice not suitable for large datasets.*
- *Another disadvantage is that SVM classifiers **do not work well with overlapping classes**.*

# 5.2.5. Support Vector Machines



- **Applications of Support Vector Machines**
- *Inverse geo sounding Problem*
- *Seismic Liquefaction Potential*
- *Protein Fold and Remote Homology Detection*
- *Data Classification*
- *Facial Expression Classification*
- *Texture Classification using*
- *Text Classification*
- *Speech Recognition*
- *Stenography Detection in Digital Images*
- *Cancer Diagnosis and Prognosis*

## 5.2.6. Time Series



- Time series is a sequence of observations of categorical or numeric variables indexed by a date, or timestamp.
- A clear example of time series data is the time series of a stock price.
- In the Table 5.2.2, we can see the basic structure of time series data.
- In this case the observations are recorded every hour.
- Normally, the first step in time series analysis is to plot the series, this is normally done with a line chart.
- The most common application of time series analysis is forecasting future values of a numeric value using the temporal structure of the data.
- This means, the available observations are used to predict values from the future.

## 5.2.6. Time Series



• Table 5.2.5.2.

| Timestamp           | Stock - Price |
|---------------------|---------------|
| 2015-10-11 09:00:00 | 100           |
| 2015-10-11 10:00:00 | 110           |
| 2015-10-11 11:00:00 | 105           |
| 2015-10-11 12:00:00 | 90            |
| 2015-10-11 13:00:00 | 120           |

## 5.2.6. Time Series



- **Time series data can be phrased as supervised learning.**
- Given a sequence of numbers for a time series dataset, we can restructure the data to look like a supervised learning problem.
- We can do this by using previous time steps as input variables and use the next time step as the output variable.
- The temporal ordering of the data, implies that traditional regression methods are not useful.
- To build robust forecast, we need models that take into account the temporal ordering of the data.

## 5.2.6. Time Series



- The most widely used model for Time Series Analysis is called ***Autoregressive Moving Average*** (ARMA).
- The model consists of two parts, an ***autoregressive*** (AR) part and a ***moving average*** (MA) part.
- The model is usually then referred to as the  $ARMA(p, q)$  model where  $p$  is the order of the autoregressive part and  $q$  is the order of the moving average part.
- The python code and output plot can be presented as follows:

## 5.2.6. Time Series

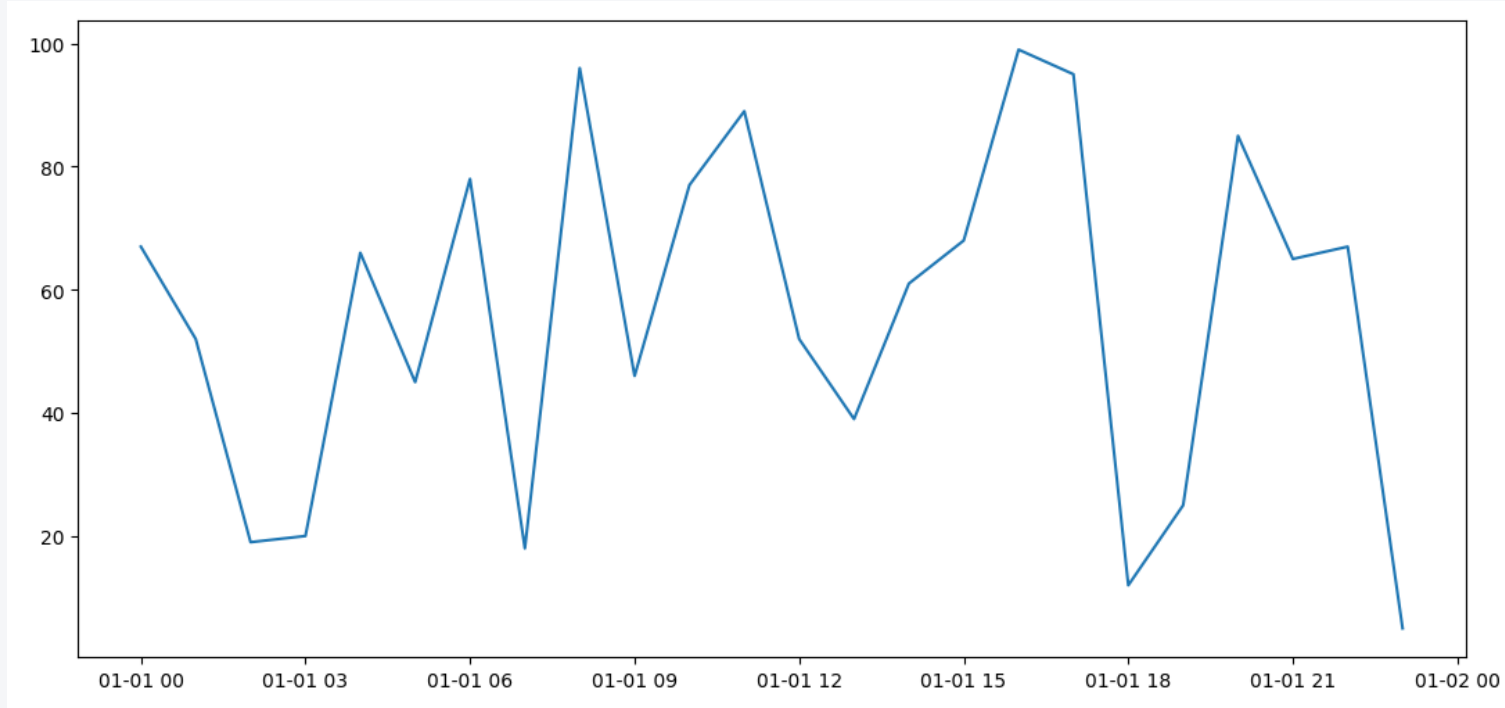


- `import matplotlib.pyplot as plt`
- `import datetime`
- `import numpy as np`
- `plt.rcParams["figure.figsize"] = [7.50, 3.50]`
- `plt.rcParams["figure.autolayout"] = True`
- `x = np.array([datetime.datetime(2021, 1, 1, i, 0) for i in range(24)])`
- `y = np.random.randint(100, size=x.shape)`
- `plt.plot(x, y)`
- `plt.show()`

## 5.2.6. Time Series



- Fig. 5.2.29.
- Output.



## 5.2.6. Time Series



- **Pros of Time Series**
- *Time series analysis helps organizations understand the underlying causes of trends or systemic patterns over time.*
- *Using data visualizations, business users can see seasonal trends and dig deeper into why these trends occur.*
- *With modern analytics platforms, these visualizations can go far beyond line graphs.*
- **Cons of Time Series**
- *Time series analysis have*
- *problems with generalization on multivariable case,*

## 5.2.6. Time Series



- *difficulty in obtaining appropriate measures,*
- *and problems with accurately identifying the correct model to represent the data.*
- **Applications**
- *The applications of time series models are manifold, including*
- *sales forecasting,*
- *weather forecasting,*
- *inventory studies etc.*
- *In decisions that involve factor of uncertainty of the future, time series models have been found one of the most effective methods of forecasting.*

## 5.2.6. Time Series



- **Autoregression**
- Autoregressive model is given by

$$X_t = c + \sum_{i=1}^P \phi_i X_{t-i} + \varepsilon_t \quad (5.2.14)$$

- where  $\{\phi_1, \dots, \phi_p\}$  are parameters to be estimated,  $c$  is a constant, and the random variable  $\varepsilon_t$  represents the white noise.
- Some constraints are necessary on the values of the parameters so that the model remains stationary.

## 5.2.6. Time Series

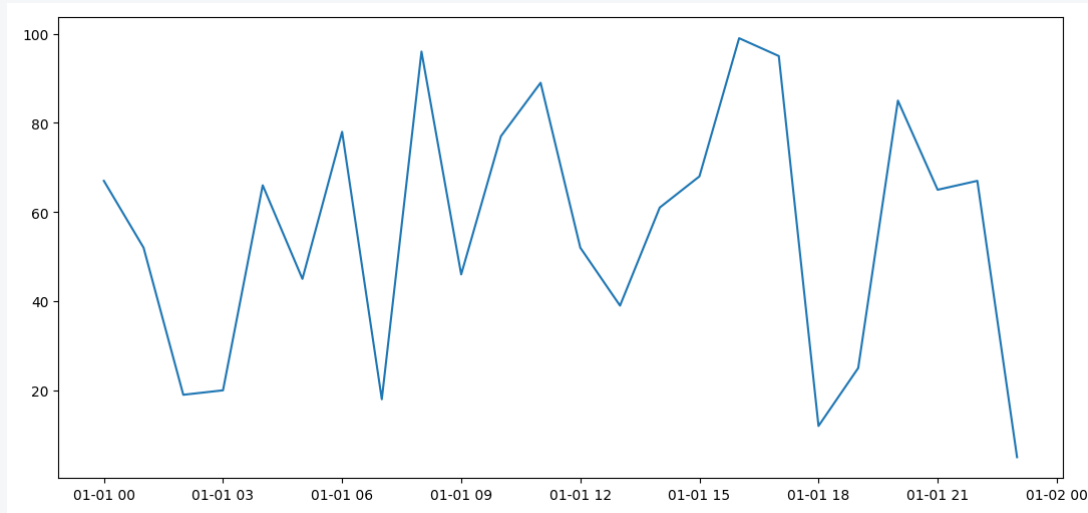


- For example, we can predict the value for the next time step ( $t+1$ ) given the observations at the two last time steps ( $t-1$  and  $t-2$ ).
- As a regression model, this would look as follows:
- $$x(t+1) = b_0 + b_1 * x(t-1) + b_2 * x(t-2). \quad (5.2.15)$$
- Because the regression model uses data from the same input variable at previous time steps, it is referred to as an autoregression (regression of self).
- The technique can be used on time series where input variables are taken as observations at previous time steps, called lag variables.

## 5.2.6. Time Series



- To find values of  $b_0$ ,  $b_1$ , and  $b_2$ , one can write
- $x(t+1) = b_0 + b_1*x(t) + b_2*x(t-1);$
- $x(t) = b_0 + b_1*x(t-1) + b_2*x(t-2);$
- $x(t-1) = b_0 + b_1*x(t-2) + b_2*x(t-3)$
- 
- and obtain  $b_0$ ,  $b_1$ , and  $b_2$
- by solving SLE given above.
- Then, (5.2.15) can be used
- to find  $X$  for any  $t$ .
- 



## 5.2.6. Time Series



- **Moving Average**
- The notation  $MA(q)$  refers to the moving average model of order  $q$  –

- $$X_t = \mu + \varepsilon_t + \sum_{i=1}^q \theta_i \varepsilon_{t-i} \quad (5.2.16)$$

- where the  $\theta_1, \dots, \theta_q$  are the parameters of the model,
- $\mu$  is the expectation of  $X_t$ ,
- and the  $\varepsilon_t, \varepsilon_{t-1}, \dots$  are white noise error terms.
- The python code and output results can be presented as follows:

## 5.2.6. Time Series



- # Python program to calculate moving average using numpy
- 
- import numpy as np
- 
- arr = [1, 2, 3, 7, 9]
- window\_size = 3
- 
- i = 0
- # Initialize an empty list to store moving averages
- moving\_averages = []

## 5.2.6. Time Series



- # Loop through the array t o
- #consider every window of size 3
- while i < len(arr) - window\_size + 1:
- 
- # Calculate the average of current window
- window\_average = round(np.sum(arr[
- i:i+window\_size]) / window\_size, 2)
- 
- # Store the average of current
- # window in moving average list
- moving\_averages.append(window\_average)
-

## 5.2.6. Time Series



- `# Shift window to right by one position`
- `i += 1`
- `print(moving_averages)`
  
- Output (*b0*, *b1*, *b2*):
- [2.20, 4.0, 6.33]

## 5.2.6. Time Series



- **Autoregressive Moving Average**
- The  $ARMA(p, q)$  model combines  $p$  autoregressive terms and  $q$  moving-average terms.
- Mathematically the model is expressed with the following formula –

- $$X_t = c + \varepsilon_t + \sum_{i=1}^P \phi_i X_{t-1} + \sum_{i=1}^q \theta_i \varepsilon_{t-i} \quad (5.2.17)$$

- We can see that the  $ARMA(p, q)$  model is a combination of  $AR(p)$  and  $MA(q)$  models.

## 5.2.6. Time Series



- To give some intuition of the model consider that the AR part of the equation seeks to estimate parameters for  $X_{t-i}$  observations to predict the value of the variable in  $X_t$ .
- It is in the end a weighted average of the past values.
- The MA section uses the same approach but with the error of previous observations,  $\varepsilon_{t-i}$ .
- So, in the end, the result of the model is a weighted average.

## 5.2.7. Random Forest Algorithm



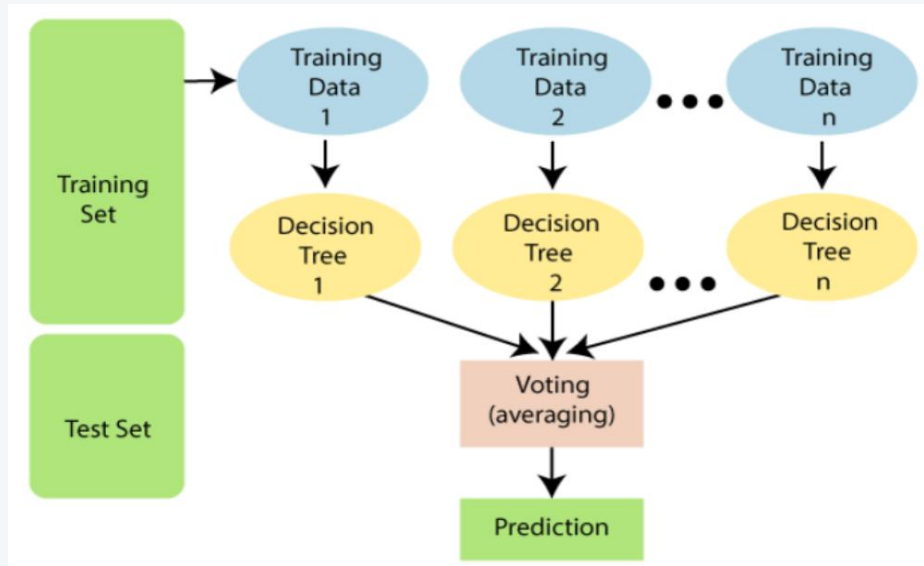
- **Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique.**
- It can be used for both Classification and Regression problems in ML.
- It is based on the concept of **ensemble learning**, which is a process of *combining multiple classifiers to solve a complex problem and to improve the performance of the model.*
- As the name suggests, ***"Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset."***
- Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.

## 5.2.7. Random Forest Algorithm



- The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.
- The below diagram explains the working of the Random Forest algorithm (Fig. 5.2.30):

Fig. 5.2.30.



## 5.2.7. Random Forest Algorithm



- **Assumptions for Random Forest**
- Since the random forest combines multiple trees to predict the class of the dataset, it is possible that some decision trees may predict the correct output, while others may not.
- But together, all the trees predict the correct output.
- Therefore, below are two assumptions for a better Random forest classifier:
- *There should be some actual values in the feature variable of the dataset so that the classifier can predict accurate results rather than a guessed result.*
- *The predictions from each tree must have very low correlations.*

## 5.2.7. Random Forest Algorithm



- **Why use Random Forest?**
- Below are some points that explain why we should use the Random Forest algorithm:
- *It takes less training time as compared to other algorithms.*
- *It predicts output with high accuracy, even for the large dataset it runs efficiently.*
- *It can also maintain accuracy when a large proportion of data is missing.*

## 5.2.7. Random Forest Algorithm



- **How does Random Forest algorithm work?**
- Random Forest works in two-phase first is to create the random forest by combining  $N$  decision tree, and second is to make predictions for each tree created in the first phase.
- The Working process can be explained in the below steps and diagram:
- *Step-1: Select random  $K$  data points from the training set.*
- *Step-2: Build the decision trees associated with the selected data points (subsets).*
- *Step-3: Choose the number  $N$  of decision trees that you want to build.*
- *Step-4: Repeat Step 1 & 5.2.*

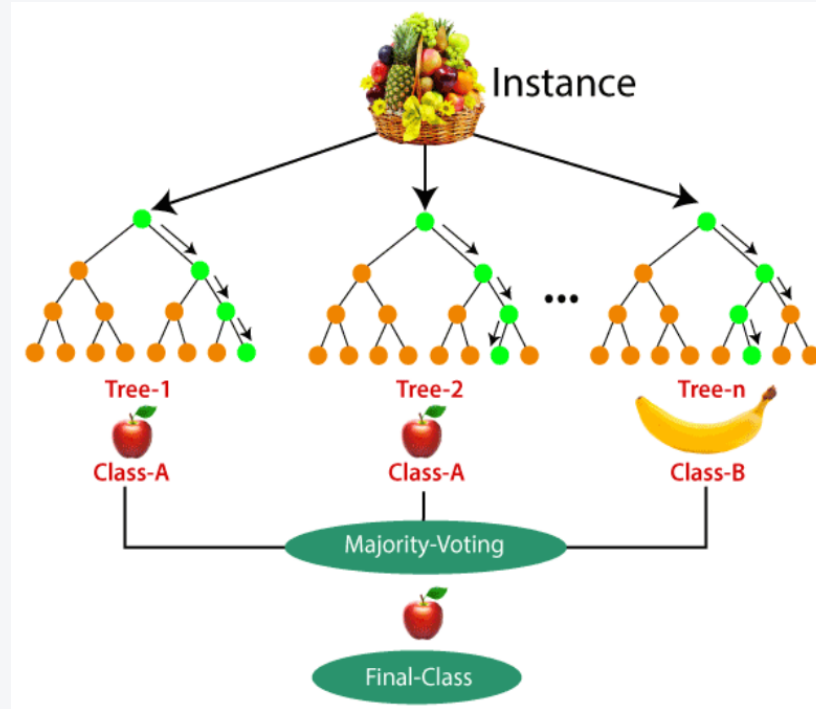
## 5.2.7. Random Forest Algorithm



- Step-5: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.
- The working of the algorithm can be better understood by the below example:
- **Example:** Suppose there is a dataset that contains multiple fruit images.
- So, this dataset is given to the Random forest classifier.
- The dataset is divided into subsets and given to each decision tree.
- During the training phase, each decision tree produces a prediction result, and when a new data point occurs, then based on the majority of results, the Random Forest classifier predicts the final decision.
- Consider the below image (Fig. 5.2.31):

## 5.2.7. Random Forest Algorithm

• Fig. 5.2.31.



## 5.2.7. Random Forest Algorithm



- **Applications of Random Forest**

- There are mainly *four sectors where Random forest mostly used:*
- *Banking:* Banking sector mostly uses this algorithm for the *identification of loan risk.*
- *Medicine:* With the help of this algorithm, *disease trends and risks of the disease can be identified.*
- *Land Use:* We can *identify the areas of similar land use* by this algorithm.
- *Marketing:* *Marketing trends can be identified* using this algorithm.

## 5.2.7. Random Forest Algorithm



- **Advantages of Random Forest**

- *Random Forest can solve both Classification and Regression tasks.*
- *It is capable of handling large datasets with high dimensionality.*
- *It enhances the accuracy of the model and prevents the overfitting issue.*

- **Disadvantages of Random Forest**

- *Although random forest can be used for both classification and regression tasks, it is more suitable for solving classification tasks.*

## 5.2.7. Random Forest Algorithm



- The python code and output results can be presented as follows:
- First, start with importing necessary Python packages –
  - `import numpy as np`
  - `import matplotlib.pyplot as plt`
  - `import pandas as pd`

## 5.2.7. Random Forest Algorithm



- Next, download the iris dataset from its weblink as follows –
- `path = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"`
- Next, we need to assign column names to the dataset as follows –
- `headers = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'Class']`
- Now, we need to read dataset to pandas dataframe as follows –
- `dataset = pd.read_csv(path, names = headers) dataset.head()`

## 5.2.7. Random Forest Algorithm



- Table 5.2.3.

|   | sepal-length | sepal-width | petal-length | petal-width | Class       |
|---|--------------|-------------|--------------|-------------|-------------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | Iris-setosa |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | Iris-setosa |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | Iris-setosa |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | Iris-setosa |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | Iris-setosa |

## 5.2.7. Random Forest Algorithm



- Data Preprocessing will be done with the help of following script lines.
- `X = dataset.iloc[:, :-1].values`
- `y = dataset.iloc[:, 4].values`
- Next, we will divide the data into train and test split. The following code will split the dataset into 70% training data and 30% of testing data –
- `from sklearn.model_selection import train_test_split`
- `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.30)`

## 5.2.7. Random Forest Algorithm



- Next, train the model with the help of *RandomForestClassifier* class of sklearn as follows –
  - `from sklearn.ensemble import RandomForestClassifier`
  - `classifier = RandomForestClassifier(n_estimators = 50)`
  - `classifier.fit(X_train, y_train)`
- At last, we need to make prediction. It can be done with the help of following script –
  - `y_pred = classifier.predict(X_test)`

## 5.2.7. Random Forest Algorithm



- Next, print the results as follows –
- `from sklearn.metrics import classification_report, confusion_matrix, accuracy_score`
- `result = confusion_matrix(y_test, y_pred)`
- `print("Confusion Matrix:")`
- `print(result)`
- `result1 = classification_report(y_test, y_pred)`
- `print("Classification Report:",)`
- `print (result1)`
- `result2 = accuracy_score(y_test,y_pred)`
- `print("Accuracy:",result2)`

## 5.2.7. Random Forest Algorithm



- **Output**
- Confusion Matrix:
- $\begin{bmatrix} 13 & 0 & 0 \\ 0 & 13 & 0 \\ 0 & 3 & 16 \end{bmatrix}$

## 5.2.7. Random Forest Algorithm



- Classification Report:
- precision recall f1-score support
- Iris-setosa            1.00    1.00    1.00    13
- Iris-versicolor      0.81    1.00    0.90    13
- Iris-virginica        1.00    0.84    0.91    19
- accuracy        0.93    45
- macro avg        0.94    0.95    0.94    45
- weighted avg    0.95    0.93    0.93    45
- *Accuracy: 0.9333333333333333*